

行程表 (暫定)

W	Date	Lecture	Homework	TA Class
1	09/10, 09/11	Course Information / No class - Registration	Homework 00 (Wed.)	V
2	09/17, 09/18	Git Introduction		
3	09/24, 09/25	OOP Concept	Homework 01 (Wed.)	V
4	10/01, 10/02	Introducing What C++ is		
5	10/08, 10/09	Class / Essential STL Introduction	Homework 02 (Wed.)	
6	10/15, 10/16	Lec03: Encapsulation		V
7	10/22, 10/23	Lec04: Inheritance	Homework 03 (Wed.)	
8	10/29, 10/30	Lec04: Inheritance		V
9	11/05, 11/06	Physical Hand-Written Midterm	Homework 04 (Mon.)	
10	11/12, 11/13	Physical Computer-based Midterm		
11	11/19, 11/20	Lec05: Polymorphism	Homework 05 (Wed.)	V
12	11/26, 11/27	Lec05: Polymorphism		
13	12/03, 12/04	Lec05: Polymorphism - LAB	Homework 06 (Wed.)	V
14	12/10, 12/11	Lec06: Composition & Interface		
15	12/17, 12/18	Lec06: Composition & Interface	Homework 07 (Wed.)	
16	12/24, 12/25	Lec06: Composition & Interface - Lab/ No class		V
17	12/31, 01/01	Physical Hand-Written Final / No class		
18	01/07, 01/08	Lec07: Experience sharing & OOPL - Preparation / Physical Computer-based Final		

物件導向程式設計

第柒章：多型

授課講師：孫勤昱博士

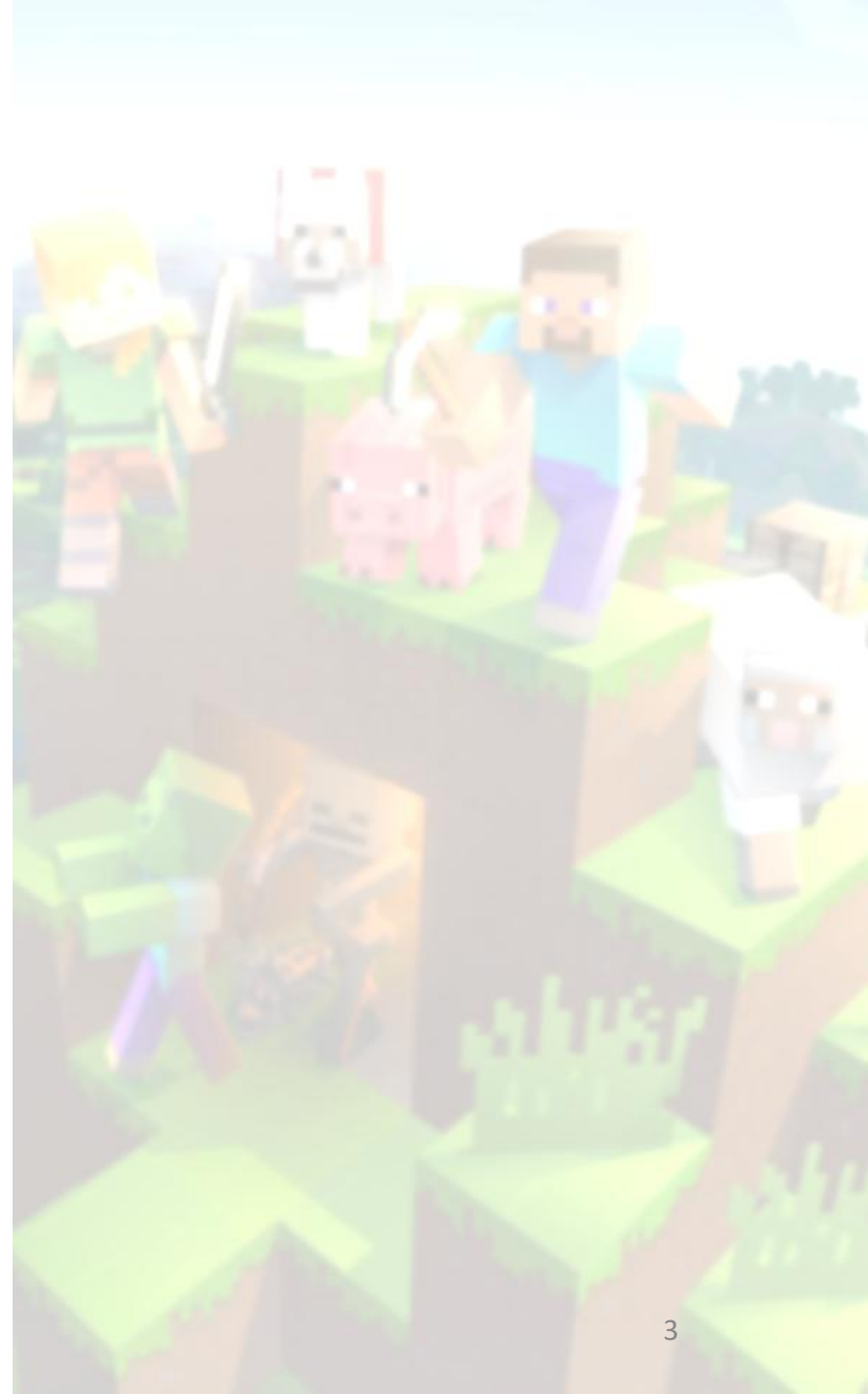
國立臺北科技大學 資訊工程系

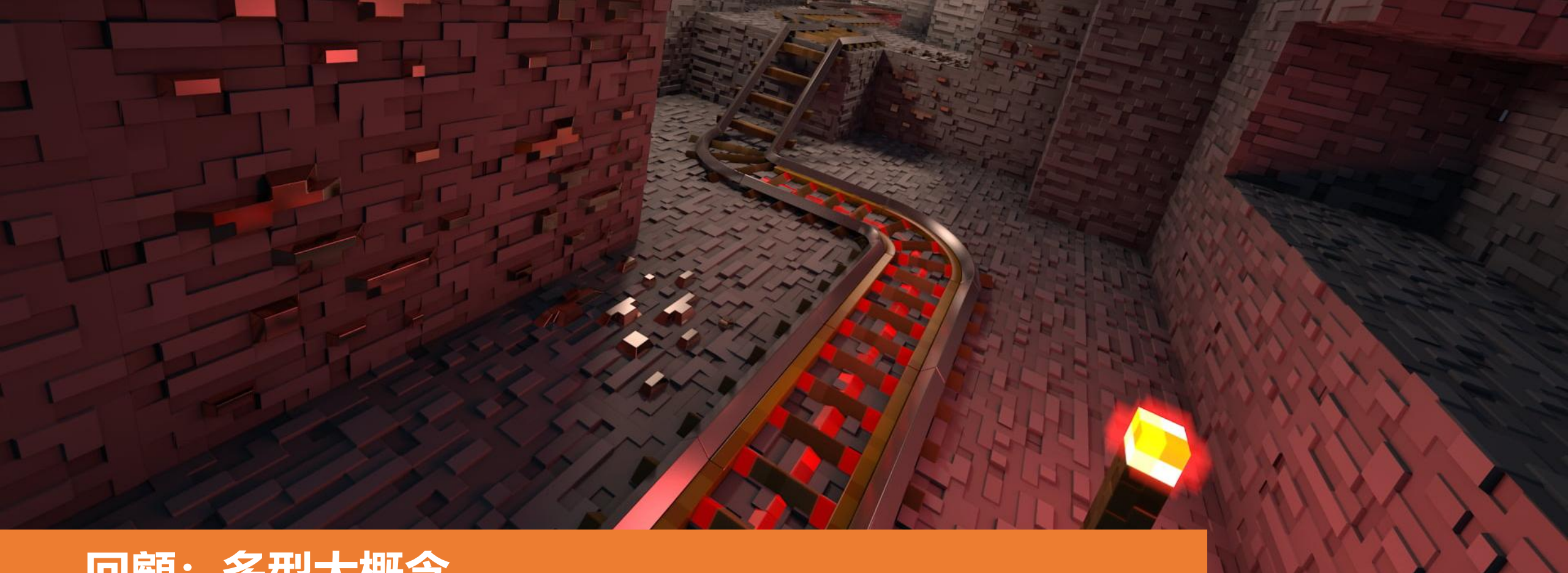
cysun@ntut.edu.tw



大綱

- 回顧：多型大概念
- 為什麼我們會需要多型
- Object Casting
- 更深入的探討 Virtual
- Virtual Destructor
- 多型種類：Inclusion
 - Function matching
- 多型種類：Ad-Hoc
 - Function Overloading
 - Operator Overloading
- 多型種類：Parametric
 - 樣板 (Template) 概念介紹





回顧：多型大概念

在接下來的課程中，我們將以 Minecraft 這款遊戲為來回顧與解釋多型

MINECRAFT

100% Pure!

單人遊戲

多人遊戲

材質包

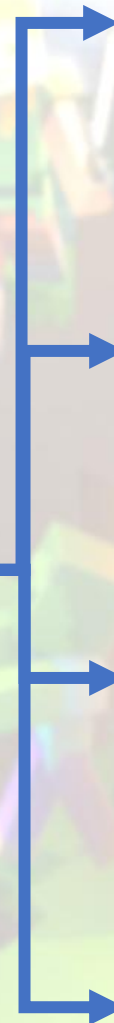
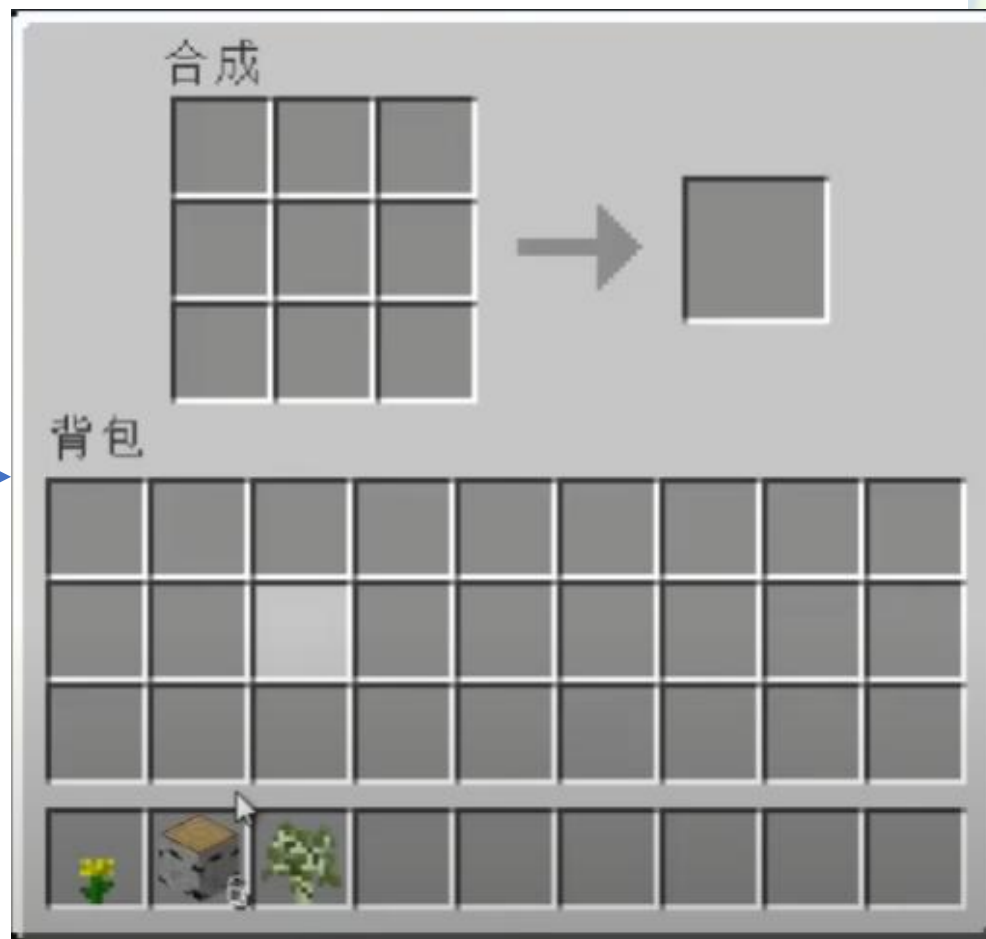


選項...

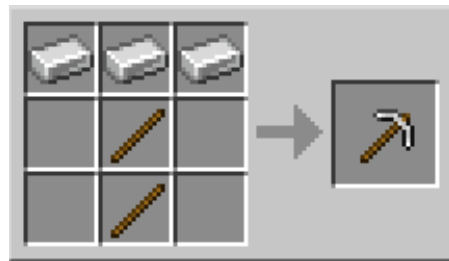
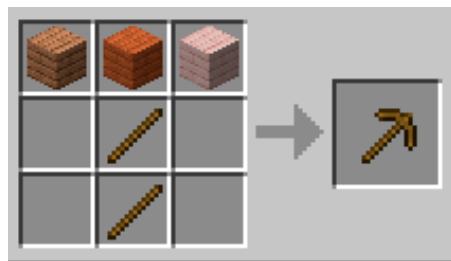
離開遊戲



合成



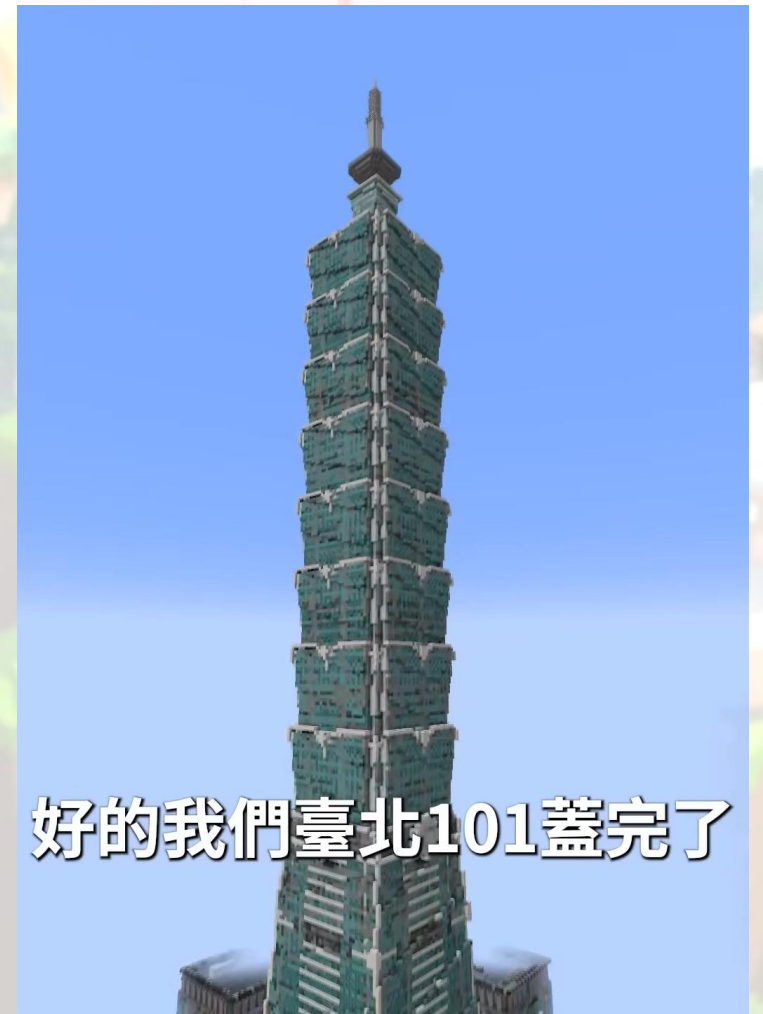
合成



物品	木鎬	金鎬	石鎬	鐵鎬	鑽石鎬
攻擊傷害	2 (♥)	2 (♥)	3 (♥♥)	4 (♥♥)	5 (♥♥♥)
攻擊速度	1.2	1.2	1.2	1.2	1.2
每秒攻擊 傷害 (DPS)	2.4 (♥ × 1.2)	2.4 (♥ × 1.2)	3.6 (♥ × 1.8)	4.8 (♥ × 2.4)	6 (♥♥♥)
可使用次 數	30	16	66	125	781
總攻擊傷 害 ^[註1]	60 (♥ × 30)	32 (♥ × 16)	198 (♥ × 99)	500 (♥ × 250)	3905 (♥ × 1952.5)

工程師/建築師/天才

- 臺北101
 - https://youtu.be/ot7dN55R_gc?t=1130
- 瀑布打字機
 - <https://youtu.be/RyQSfWNE21o?t=285>
- 時鐘
 - <https://youtu.be/RyQSfWNE21o?t=357>
- 紅石影印機 + Email系統
 - <https://youtu.be/RyQSfWNE21o?t=412>
- 在Minecraft裡面玩Minecraft
 - <https://youtu.be/5YADlrSJUqA?t=96>



回顧：多型大概念

- 考慮正在設計 Minecraft 的遊戲細節，我們來設計「挖礦物」這個動作

Coal Ore
(煤礦)



Coal
(煤炭)

Iron Ore
(鐵礦)



Raw Iron
(鐵原礦)

Gold Ore
(金礦)



Raw Gold
(金原礦)

Redstone Ore
(紅石礦)



Redstone Dust
(紅石粉)

Diamond Ore
(鑽石礦)

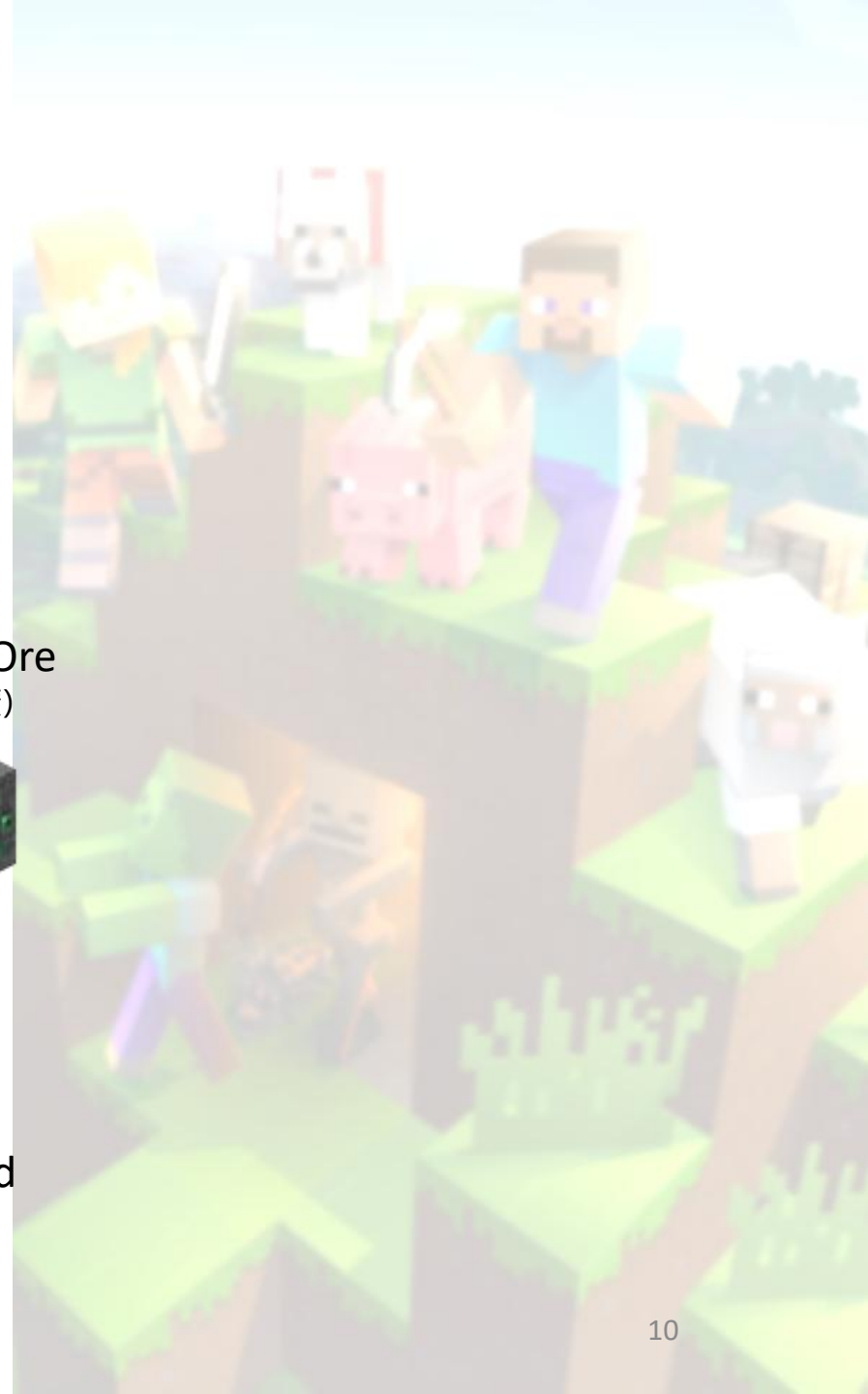


Diamond
(鑽石)

Emerald Ore
(綠寶石礦)



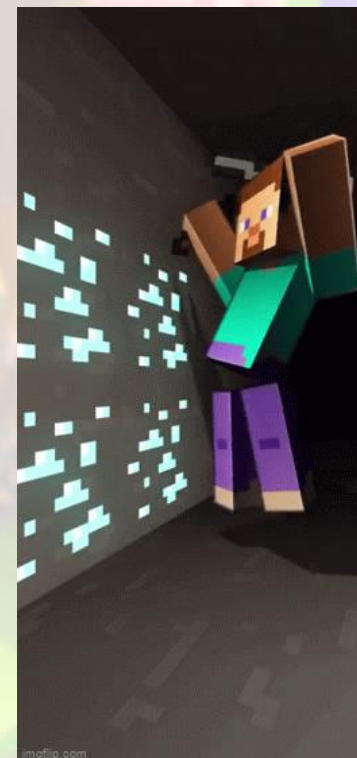
Emerald
(綠寶石)



回顧：多型大概念

- 在 Minecraft 遊戲中，當玩家挖掘礦物時（上排），玩家會得到對應的掉落產物（下排）
- 例如：當你挖鑽石礦時，挖掉後你會得到一枚鑽石

礦物						
Coal Ore (煤礦)	Iron Ore (鐵礦)	Gold Ore (金礦)	Redstone Ore (紅石礦)	Diamond Ore (鑽石礦)	Emerald Ore (綠寶石礦)	
						
						
產物	Coal (煤炭)	Raw Iron (鐵原礦)	Raw Gold (金原礦)	Redstone Dust (紅石粉)	Diamond (鑽石)	Emerald (綠寶石)



回顧：多型大概念

- 回顧之前的課程，我們可以直覺想到能夠用一個 Ore 的類別來當作父類，然後有一個 GetOre() 的虛擬函數
- 對於這些虛擬函數，我們可以交由子類來定義他們應該掉哪種礦物下來，或者就不要掉東西下來

Coal Ore
(煤礦)



Coal
(煤炭)

Iron Ore
(鐵礦)



Raw Iron
(鐵原礦)

Gold Ore
(金礦)



Raw Gold
(金原礦)

Redstone Ore
(紅石礦)



Redstone Dust
(紅石粉)

Diamond Ore
(鑽石礦)



Diamond
(鑽石)

Emerald Ore
(綠寶石礦)



Emerald
(綠寶石)

```
1 class Ore {  
2     virtual OreResult GetOre();  
3 };  
4  
5 class IronOre : public Ore {  
6     OreResult GetOre() override;  
7 };
```

回顧：多型大概念

- 我們嘗試搞定了挖掘掉東西的部分，但我們接下來要處理玩家挖掘的部分

Coal Ore
(煤礦)



Iron Ore
(鐵礦)



Gold Ore
(金礦)



Redstone Ore
(紅石礦)



Diamond Ore
(鑽石礦)



Emerald Ore
(綠寶石礦)



Coal
(煤炭)



Raw Iron
(鐵原礦)



Raw Gold
(金原礦)



Redstone Dust
(紅石粉)



Diamond
(鑽石)

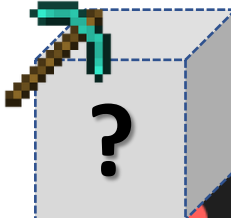


Emerald
(綠寶石)



```
1  if(type == "IronOre"){
2      ironOre.GetOre();
3  }
4
5  if(type == "Coal"){
6      coalOre.GetOre();
7  }
8
9  if(type == "Redstone"){
10     redstoneOre.GetOre();
11 }
12
13 /* ... */
```

問題討論



```
1 class Ore {
2     virtual OreResult GetOre();
3 }
4
5 class IronOre : public Ore {
6     OreResult GetOre() override;
7 }
```

```
1  if(type == "IronOre"){
2      ironOre.GetOre();
3  }
4
5  if(type == "Coal"){
6      coalOre.GetOre();
7  }
8
9  if(type == "Redstone"){
10     redstoneOre.GetOre();
11 }
12
13 /* ... */
```

回顧：多型大概念

- 我們嘗試搞定了挖掘掉東西的部分，但我們接下來要處理玩家挖掘的部分
- 然後你會發現會長得像這樣...
- 如果今天遊戲中有 100 種礦物，我就會需要 100 個 if 來處理礦物的部分
- 這樣聽起來不太妙



```
1  if(type == "IronOre"){
2      ironOre.GetOre();
3  }
4
5  if(type == "Coal"){
6      coalOre.GetOre();
7  }
8
9  if(type == "Redstone"){
10     redstoneOre.GetOre();
11 }
12
13 /* ... */
```

礦物

Coal Ore
(煤礦)



Iron Ore
(鐵礦)



Gold Ore
(金礦)



Redstone Ore
(紅石礦)



Diamond Ore
(鑽石礦)



Emerald Ore
(綠寶石礦)



產物

Coal
(煤炭)



Raw Iron
(鐵原礦)



Raw Gold
(金原礦)



Redstone Dust
(紅石粉)



Diamond
(鑽石)

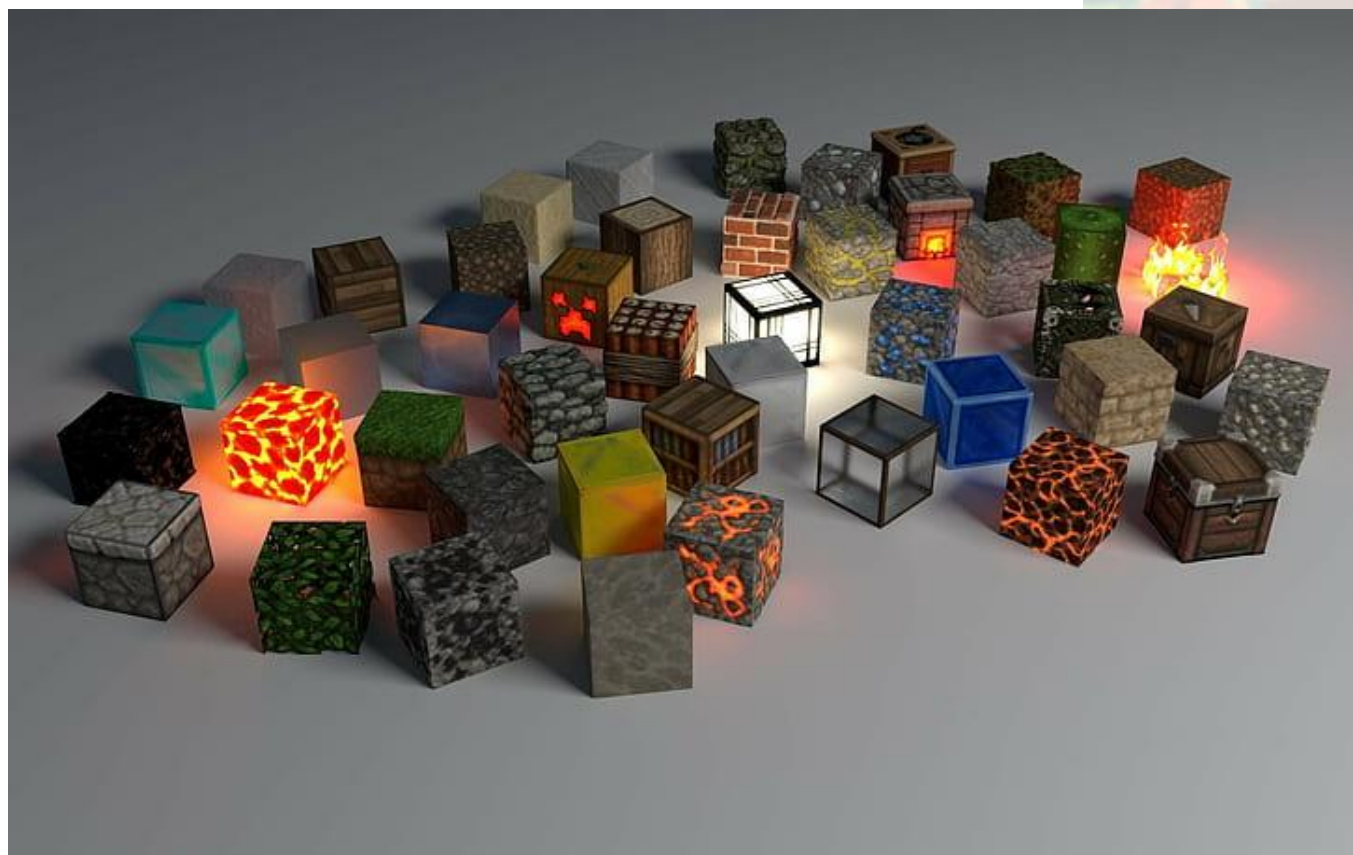


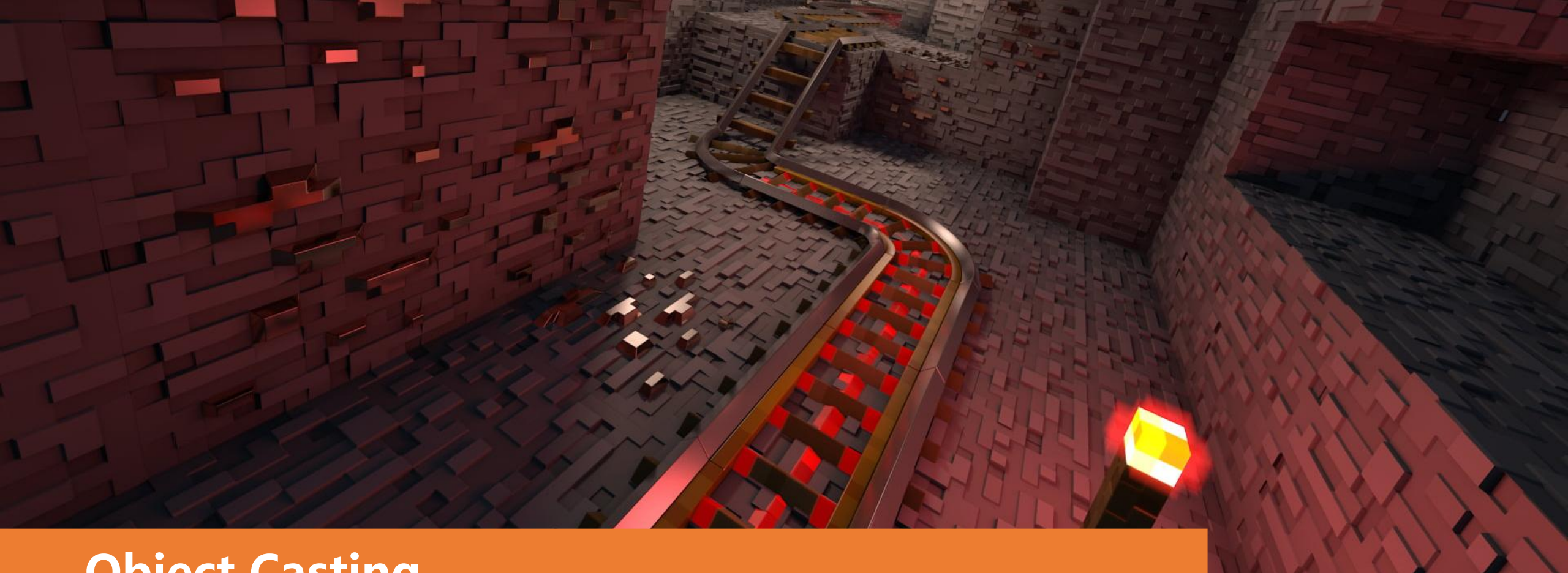
Emerald
(綠寶石)



回顧：多型大概念

- 所以要想一個辦法，可以應付當有 100 個礦物時，能夠正常運作又不會產生大量實作來增加維護難度



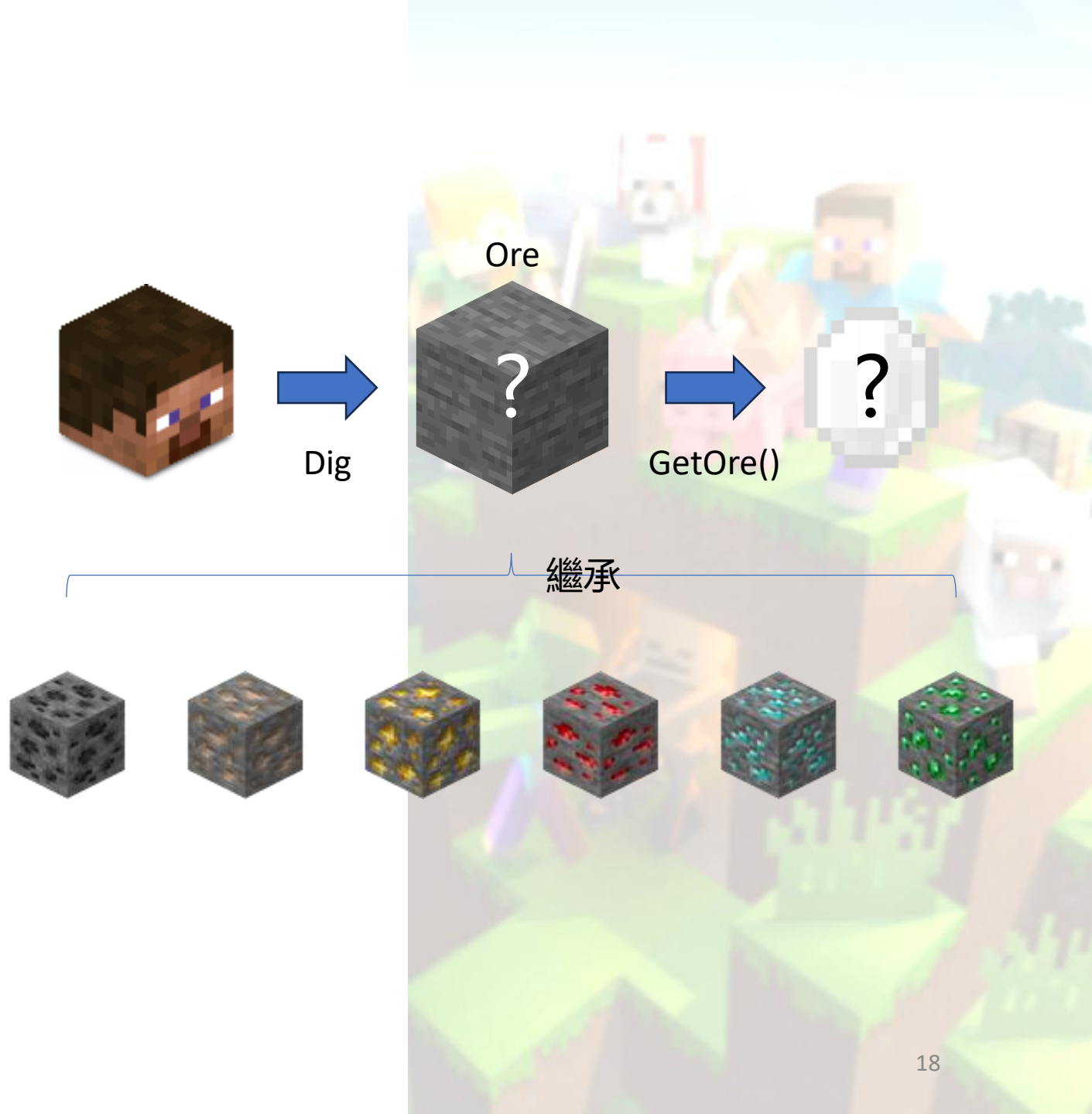


Object Casting

以紅石礦物為例，怎麼體現紅石礦變成礦石的過程呢？

觀察

- 由於這些礦物全部都會掉落礦石，所以我們可以預期他們**都會有一個** GetOre() 的 function
- 如果今天使用者挖掘任意一個礦石時，我們可以先不管他是哪一種礦物，直接將其視為一個「廣泛的礦石 (Ore)」
 - 使用繼承 (當作父類)
 - 一樣要Virtual + Override
 - 直接呼叫 Ore 底下子類別的 GetOre() 來取得礦物

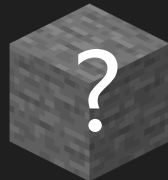


繼承/Virtual/Override

- 我們有了一個礦（廣泛的礦石）的 class 與紅石礦的 class



```
1  #ifndef ORE_STONE_HPP
2  #define ORE_STONE_HPP
3
4  class Ore{
5  public:
6      virtual std::string GetOre(){
7          return "Unknown Ore";
8      }
9  };
10
11 #endif
```



```
1  #ifndef OOP_REDSTONEORESTONE_H
2  #define OOP_REDSTONEORESTONE_H
3
4  #include "OreStone.hpp"
5
6  class RedStoneOreStone : public Ore{
7  public:
8      std::string GetOre() override {
9          return "Red Stone";
10     }
11
12     void ShineStoneTexture(){
13         /* Do something */
14     }
15 };
16
17 #endif // OOP_REDSTONEORESTONE_H
18
```



Output結果

- 然後我們就能夠用這個方式來取得紅石礦的礦石



```
1  #include <iostream>
2
3  #include "RedStoneOreStone.h"
4
5  int main(){
6      RedStoneOreStone redStoneOreStone;
7
8      std::cout << redStoneOreStone.GetOre() << std::endl;
9  }
```

GetOre()



GetOre() override



“Red Stone”

- 由於我們 Override 了原本 OreStone 的 GetOre(), 所以我們得到了 “Red Stone” 的輸出而不是 “Unknown Ore”

物件實作 vs 指標實作

```
1 #include <iostream>
2 #include <memory>
3
4 int main(){
5     RedStoneOreStone redStoneOreStone;
6     std::cout << redStoneOreStone.GetOre() << std::endl;
7 }
```

```
1 #include <iostream>
2 #include <memory>
3
4 int main(){
5     std::shared_ptr<RedStoneOreStone> stone = std::make_shared<RedStoneOreStone>();
6     std::cout << stone->GetOre() << std::endl;
7 }
```

兩種差異

```
1 #include <iostream>
2 #include <memory>
3
4 int main(){
5     std::shared_ptr<RedStoneOreStone> stone = std::make_shared<RedStoneOreStone>();
6     std::cout << stone->GetOre() << std::endl;
7 }
```

```
1 #include <iostream>
2 #include <Memory>
3
4 int main(){
5     std::shared_ptr<Ore> stone = std::make_shared<RedStoneOreStone>();
6     std::cout << stone->GetOre() << std::endl;
7 }
```

- Object Casting: 將原本的 Derived Class 視為 Base Class 的動作
 - 註: 也有一派的人說這是 Object Slicing, 不過我們認為 Object Slicing 比較像負面情形, 所以用 Object Casting

Object Casting

```
#include <iostream>
#include <memory>

#include "RedStoneOreStone.h"

int main(){
    std::shared_ptr<Ore> stone = std::make_shared<RedStoneOreStone>();

    std::cout << stone->GetOre() << std::endl; ➡ 
}
```

- Object Casting: 將原本的 Derived Class 視為 Base Class 的動作
 - 註: 也有一派的人說這是 Object Slicing, 不過我們認為 Object Slicing 比較像負面情形, 所以用 Object Casting

Object Casting

- 透過指標替換，方便替換成我們「想要的東西」

```
#include <iostream>
#include <memory>

#include "RedStoneOreStone.h"

int main(){
    std::shared_ptr<Ore> stone = std::make_shared<RedStoneOreStone>();

    std::cout << stone->GetOre() << std::endl;
}
```



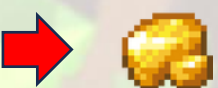
<DiamondOreStone>



<IronOreStone>



<GoldOreStone>



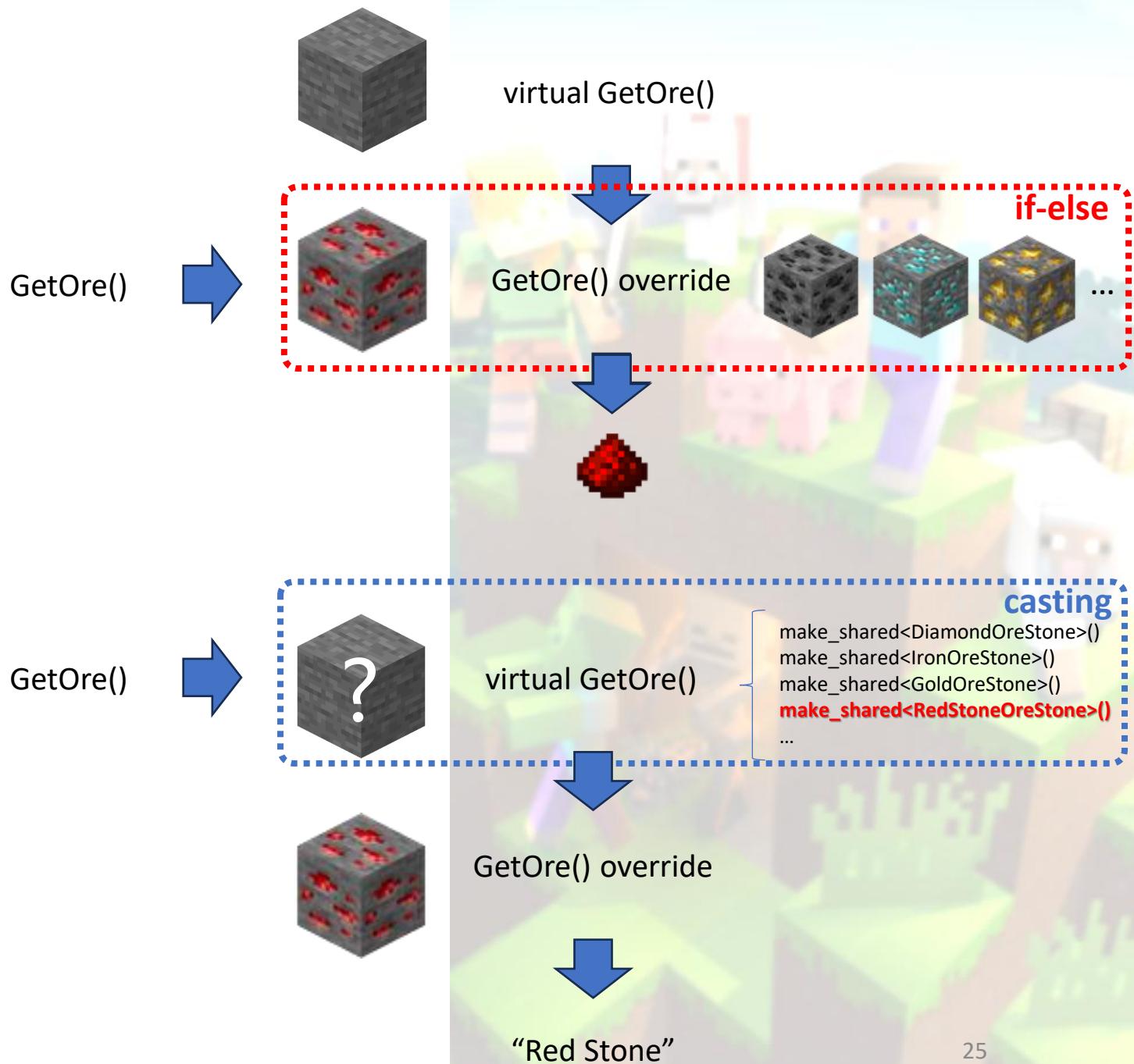
⋮

24

⋮

結論

- 為什麼我們會需要 Object Casting?
 - 如果我們有辦法可以讓 GetOre() 使用物件實際 Derived 的 GetOre(), 但類別依然是 Base Class
 - 這樣我們能夠省掉用 if-else 判斷礦物的時間, 直接 Casting 後取 GetOre() 的結果就行
- 我們需要先明白 Object Casting的觀念, 後面介紹多型的實作時, 需要具備Casting與指標/Reference的觀念

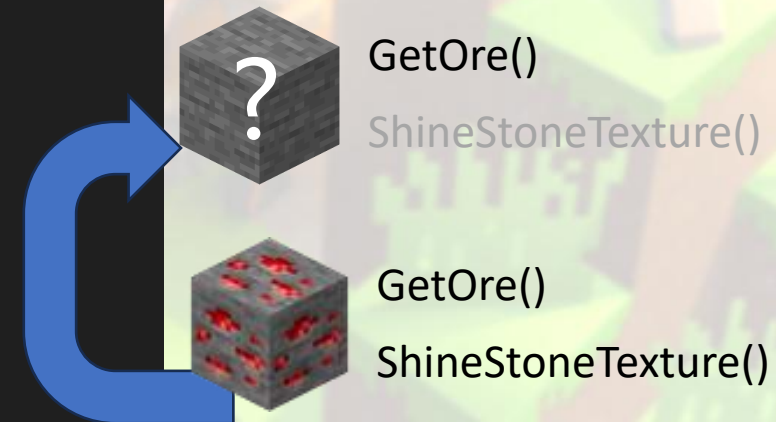


Object Casting 的問題

- 使用這樣的方式下：
 - 所有 RedStoneOreStone 額外的函式與成員都會因為 Casting 而導致子類衍生成員與變數消失
 - 僅存父類 OreStone 的成員與函式

```
1  #ifndef ORE_STONE_HPP
2  #define ORE_STONE_HPP
3
4  class OreStone {
5  public:
6      virtual std::string GetOre(){
7          return "Unknown Ore";
8      }
9  };
10
11 #endif
```

```
4  #include "OreStone.hpp"
5
6  class RedStoneOreStone : public OreStone {
7  public:
8      std::string GetOre() override {
9          return "Red Stone";
10     }
11
12     void ShineStoneTexture(){
13         /* Do something */
14     }
15 };
16
17 #endif // OOP_REDSTONEORESTONE_H
18
```



Object Casting 的問題

```
#include <iostream>
#include <memory>

#include "RedStoneOreStone.h"

int main(){
    std::shared_ptr<OreStone> stone = std::make_shared<RedStoneOreStone>();

    std::cout << stone->ShineStoneTexture() << std::endl;
}

```



```
#include <iostream>
#include <memory>

#include "RedStoneOreStone.h"

int main(){
    std::shared_ptr<OreStone> stone = std::make_shared<RedStoneOreStone>();

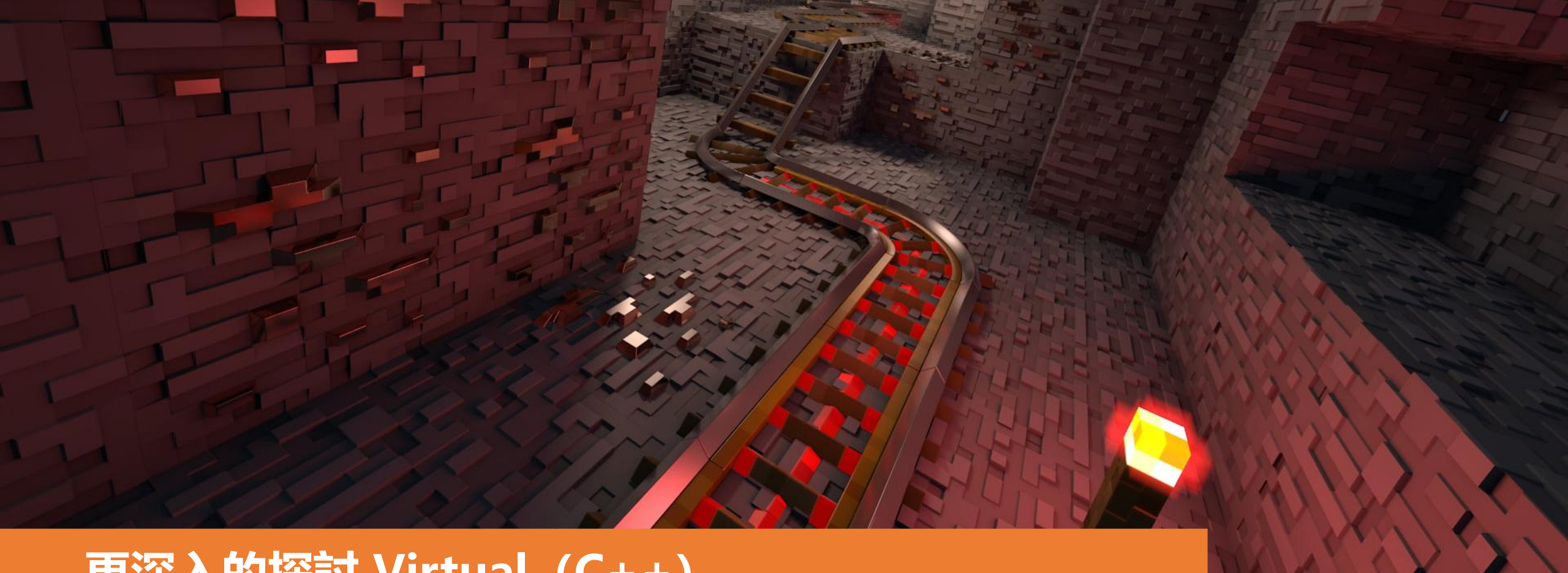
    std::cout << stone->GetOre() << std::endl;
}

```



沒有辦法透過這樣來呼叫 RedStoneOreStone 的 ShineStoneTexture()

- stone 變數所有的 function signature 全部視(退)為(化) OreStone 的 function signature



更深入的探討 Virtual (C++)

接下來，我們將探討 Late binding 與 Early binding，探討 Virtual 如何幫助這一件事情發生^註

- ^註 Early Binding 與 Late Binding 為 C++ 的語法實作細節

更深入的探討 Virtual (C++)

- 回來這兩個類別，我們會慣例在子類能夠 override 的 function 上加上 virtual

```
1  #ifndef ORE_STONE_HPP
2  #define ORE_STONE_HPP
3
4  class OreStone {
5  public:
6      virtual std::string GetOre(){
7          return "Unknown Ore";
8      }
9  };
10
11 #endif
```

```
1  #ifndef OOP_REDSTONEORESTONE_H
2  #define OOP_REDSTONEORESTONE_H
3
4  #include "OreStone.hpp"
5
6  class RedStoneOreStone : public OreStone {
7  public:
8      std::string GetOre() override {
9          return "Red Stone";
10     }
11
12     void ShineStoneTexture(){
13         /* Do something */
14     }
15 };
16
17 #endif // OOP_REDSTONEORESTONE_H
18
```

更深入的探討 Virtual (C++)

- 如果不加呢?

```
1  #ifndef ORE_STONE_HPP
2  #define ORE_STONE_HPP
3
4  class OreStone {
5  public:
6      std::string GetOre(){
7          return "Unknown Ore";
8      }
9  };
10
11 #endif
```

- 選項A 不能編譯
- 選項B Unknown Ore

```
1  #ifndef OOP_REDSTONEORESTONE_H
2  #define OOP_REDSTONEORESTONE_H
3
4  #include "OreStone.hpp"
5
6  class RedStoneOreStone : public OreStone {
7  public:
8      std::string GetOre() override {
9          return "Red Stone";
10     }
11
12     void ShineStoneTexture(){
13         /* Do something */
14     }
15 };
16
17 #endif // OOP_REDSTONEORESTONE_H
18
```

更深入的探討 Virtual (C++)

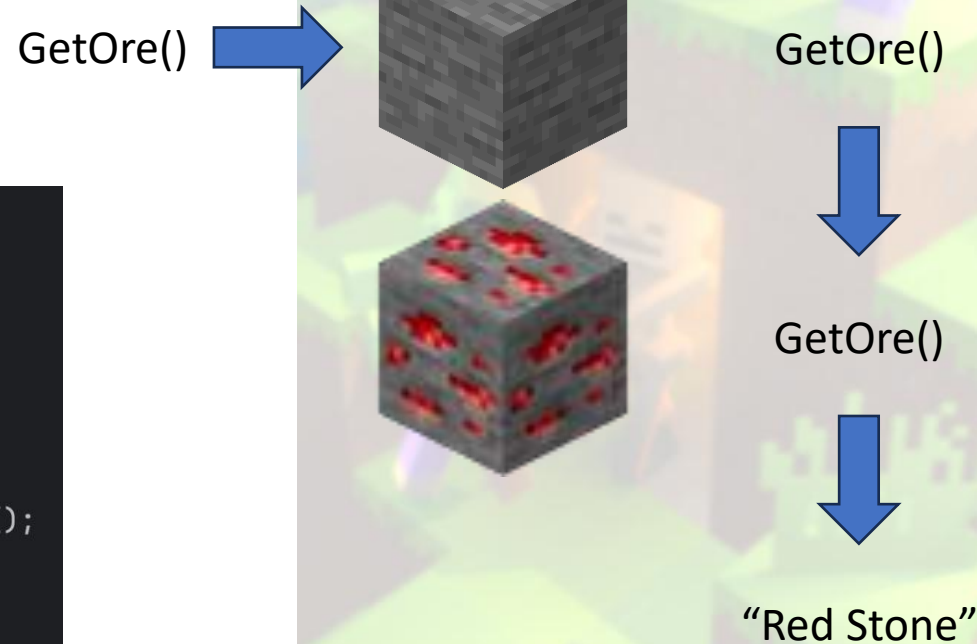
- 我們繼續前一個例子：一樣的 Object Casting 並且嘗試取得 RedStoneOreStone 的 GetOre()
- 在不加的情況時：

```
#include <iostream>
#include <memory>

#include "RedStoneOreStone.h"

int main(){
    std::shared_ptr<OreStone> stone = std::make_shared<RedStoneOreStone>();

    std::cout << stone->GetOre() << std::endl;
}
```



更深入的探討 Virtual (C++)

- 這時候你發現了輸出變成了 Unknown Ore 了，你抓不到了 Derived Class 的 GetOre()

```
#include <iostream>
#include <memory>

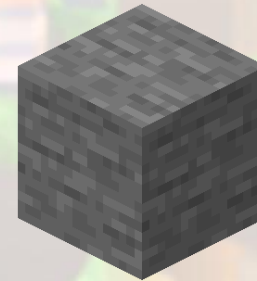
#include "RedStoneOreStone.h"

int main(){
    std::shared_ptr<OreStone> stone = std::make_shared<RedStoneOreStone>();

    std::cout << stone->GetOre() << std::endl;
}
```

```
/Users/xuan/Documents/00P2023f_example_code/cmake-build-debug/ut_all --gtest_filter=* --gtest_color=no
Testing started at 01:36 ...
Unknown Ore
Process finished with exit code 0
```

GetOre()



GetOre()



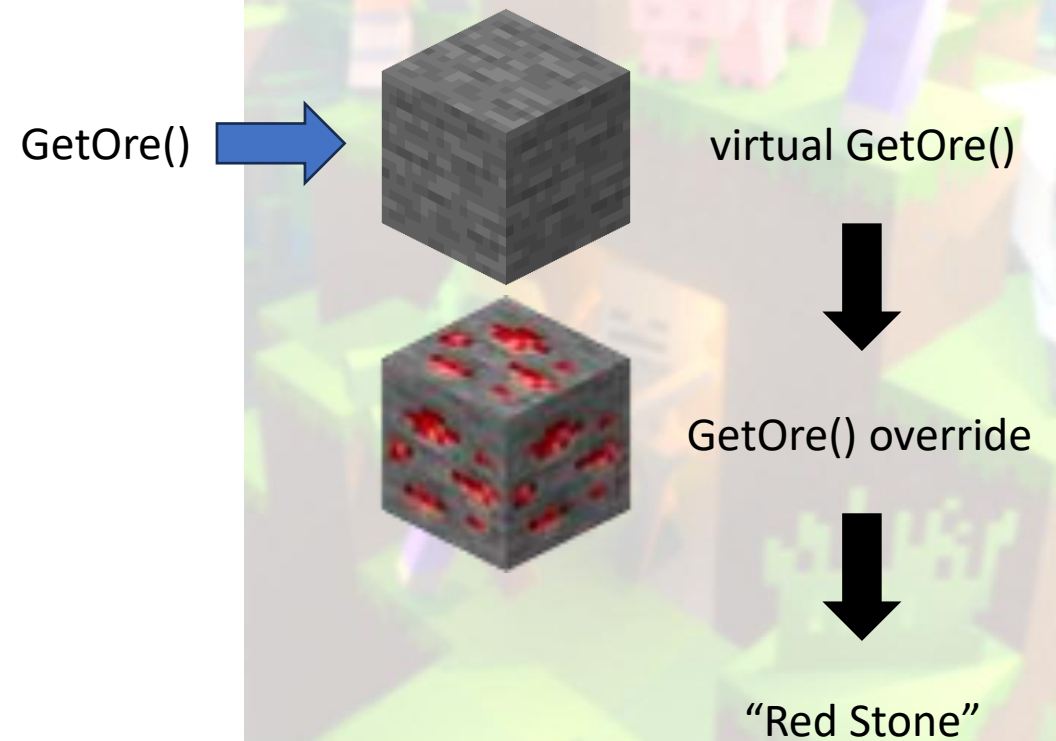
GetOre()



"Red Stone"

更深入的探討 Virtual (C++)

- 為什麼少一個 virtual 會造成無法取得 Derived Class 的 GetOre() 的情形?
- 我們將函式綁定 (Binding) 分成兩個不同的種類:
 - Early Binding: 在 **Compilation Time** 時進行綁定
 - Late Binding: 在 **Execution Time** 時進行綁定

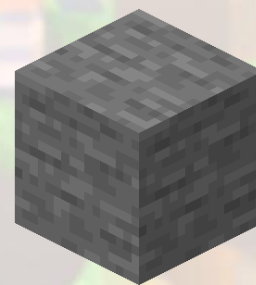


- 註: Early Binding 與 Late Binding 為 C++ 的語法實作細節

更深入的探討 Virtual (C++)

- 為什麼少一個 virtual 會造成無法取得 Derived Class 的 GetOre() 的情形?
- 我們將函式綁定 (Binding) 分成兩個不同的種類:
 - Early Binding: 在 **Compilation Time** 時進行綁定
 - Late Binding: 在 **Execution Time** 時進行綁定
- 當我們並沒有特別加上 virtual 時, 所有的函式綁定皆視為 Early Binding
 - 在 Compilation Time 時直接綁定 Base Class 的定義

GetOre()



GetOre()



GetOre()

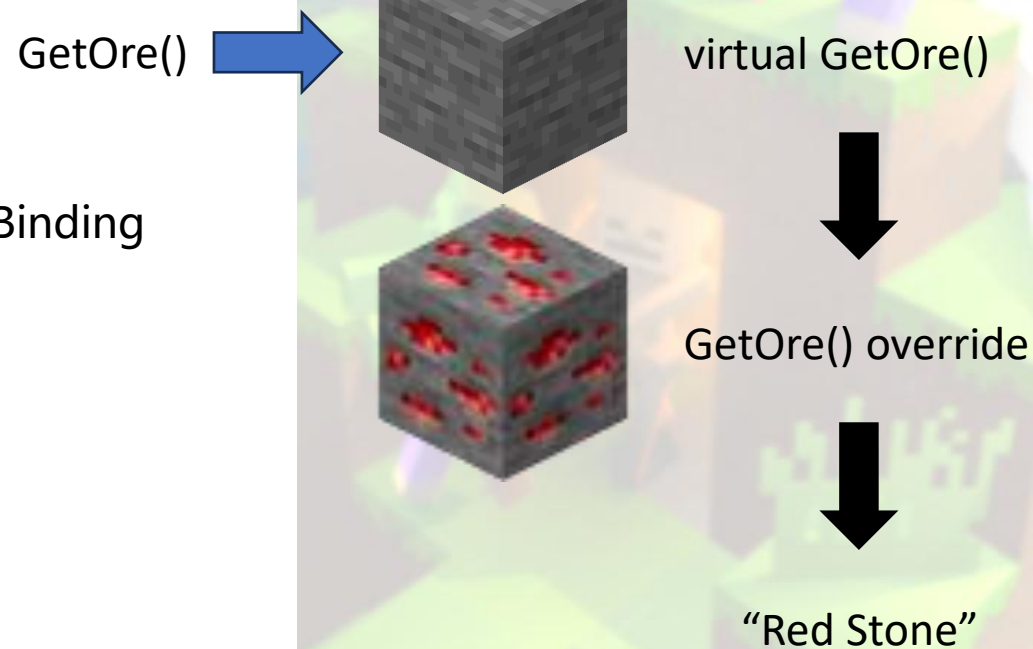


"Red Stone"

- 註: Early Binding 與 Late Binding 為 C++ 的語法實作細節

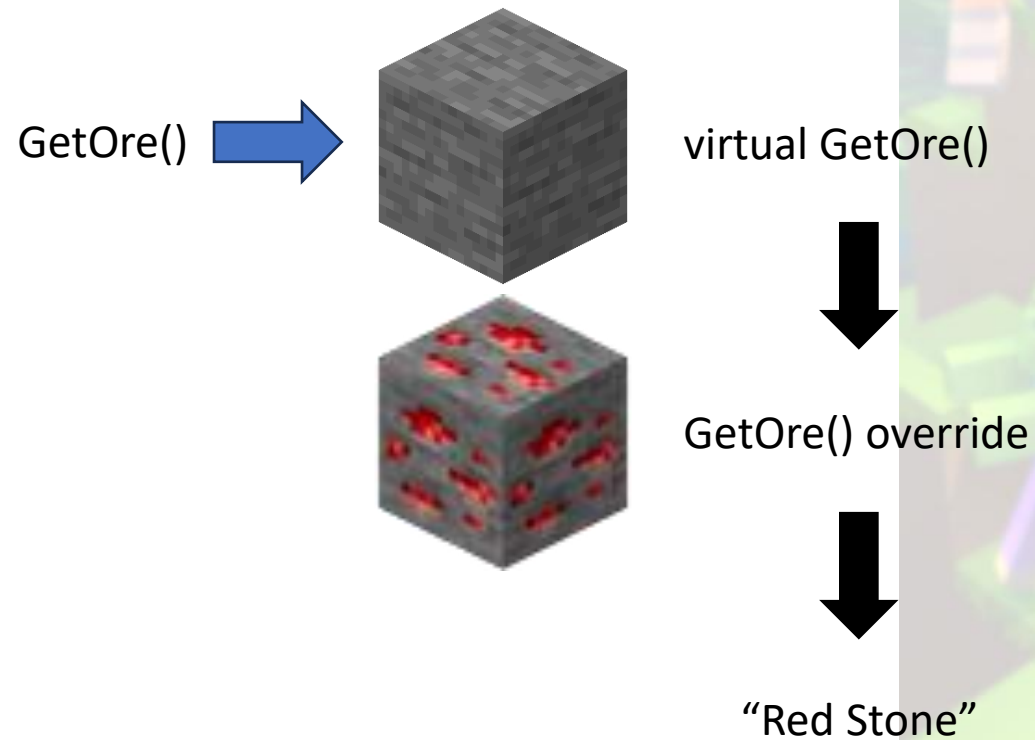
更深入的探討 Virtual (C++)

- 為什麼少一個 virtual 會造成無法取得 Derived Class 的 GetOre() 的情形?
- 我們將函式綁定 (Binding) 分成兩個不同的種類:
 - Early Binding: 在 **Compilation Time** 時進行綁定。
 - Late Binding: 在 **Execution Time** 時進行綁定。
- 當我們並沒有特別加上 virtual 時, 所有的函式綁定皆視為 Early Binding
 - 在 Compilation Time 時直接綁定 Base Class 的定義
- 當我們加上 virtual 時, 函式綁定視為 Late Binding
 - 在 Execution Time 時尋找 Function 的定義
- 註: Early Binding 與 Late Binding 為 C++ 的語法實作細節



更深入的探討 Virtual (C++)

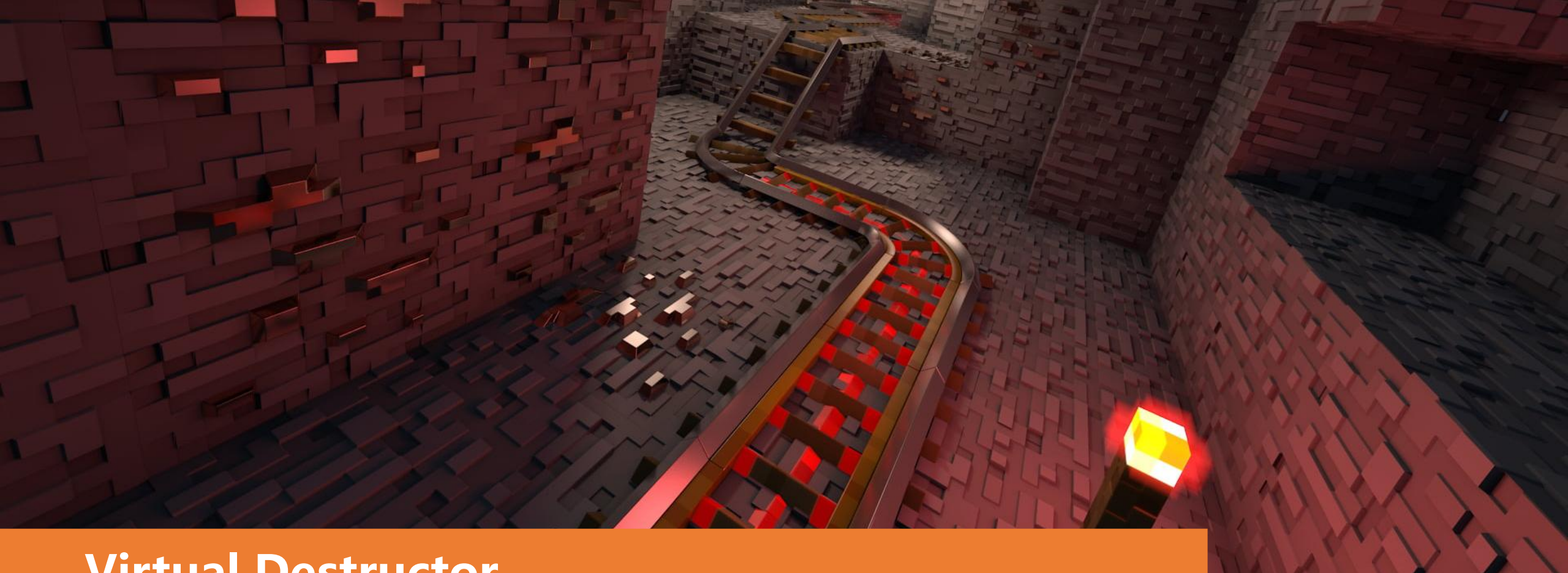
- 藉由 Virtual 的 Late Binding，我們可以使物件被呼叫時使用 Late Binding 來找到子類的定義



語意與行為上的解釋

- 上述幾頁的解釋方式是基於「C++ 語言」的一個詮釋，不同語言有不一樣的實作與解釋方式
- 語意上，我們將一個 function 前面標記 virtual 這個字，表示程式設計師與編譯器達成下列共識：
 - 這個 function 有可能有子類的實作
 - 你需要在執行時候去找他





Virtual Destructor

在我們完成了更深入的 Virtual 介紹後，我們額外想要提到另一個議題：Virtual Destructor

解構子 (Destructor)

- 回顧一下物件的生命週期



使用解構子釋放

使用建構子創立

```
1  #include "laptop.hpp"
2
3  #include <string>
4
5  Laptop::Laptop(std::string operatingSystem, double screenSize, bool isOLED,
6                std::string name, std::string cpuName, bool haveNumberKey){
7      this->operatingSystem = operatingSystem;
8      this->screenSize = screenSize;
9      this->isOLED = isOLED;
10     this->name = name;
11     this->cpuName = cpuName;
12     this->haveNumberKey = haveNumberKey;
13 }
14
15 Laptop::~Laptop(){
16 }
17
18 std::string Laptop::GetOpeartingSystem(){
19     return operatingSystem;
20 }
21
22 double Laptop::GetScreenSize(){
23     return screenSize;
24 }
25
26 bool Laptop::IsOLED(){
27     return isOLED;
28 }
29
30 std::string Laptop::GetName(){
31     return name;
32 }
33
34 std::string Laptop::GetCPUName(){
35     return cpuName;
36 }
37
38 bool Laptop::HaveNumberKey(){
39     return haveNumberKey;
40 }
```

解構子 (Destructor)

- 在 C++ 中，我們具有「解構鏈 (Destructor Chain)」註
- 對於一個具有衍伸類別的物件來說，會從最底下的子類的解構子一路釋放到最上面父類的解構子

```
#ifndef ORE_STONE_HPP
#define ORE_STONE_HPP

class OreStone {
public:
    std::string GetOre(){
        return "Unknown Ore";
    }

    virtual ~OreStone(){

    }
};

#endif
```

```
#ifndef OOP_REDSTONEORESTONE_H
#define OOP_REDSTONEORESTONE_H

#include "OreStone.hpp"

class RedStoneOreStone : public OreStone {
public:
    std::string GetOre() {
        return "Red Stone";
    }

    ~RedStoneOreStone() override{

    }

    void ShineStoneTexture(){
        /* Do something */
    }
};

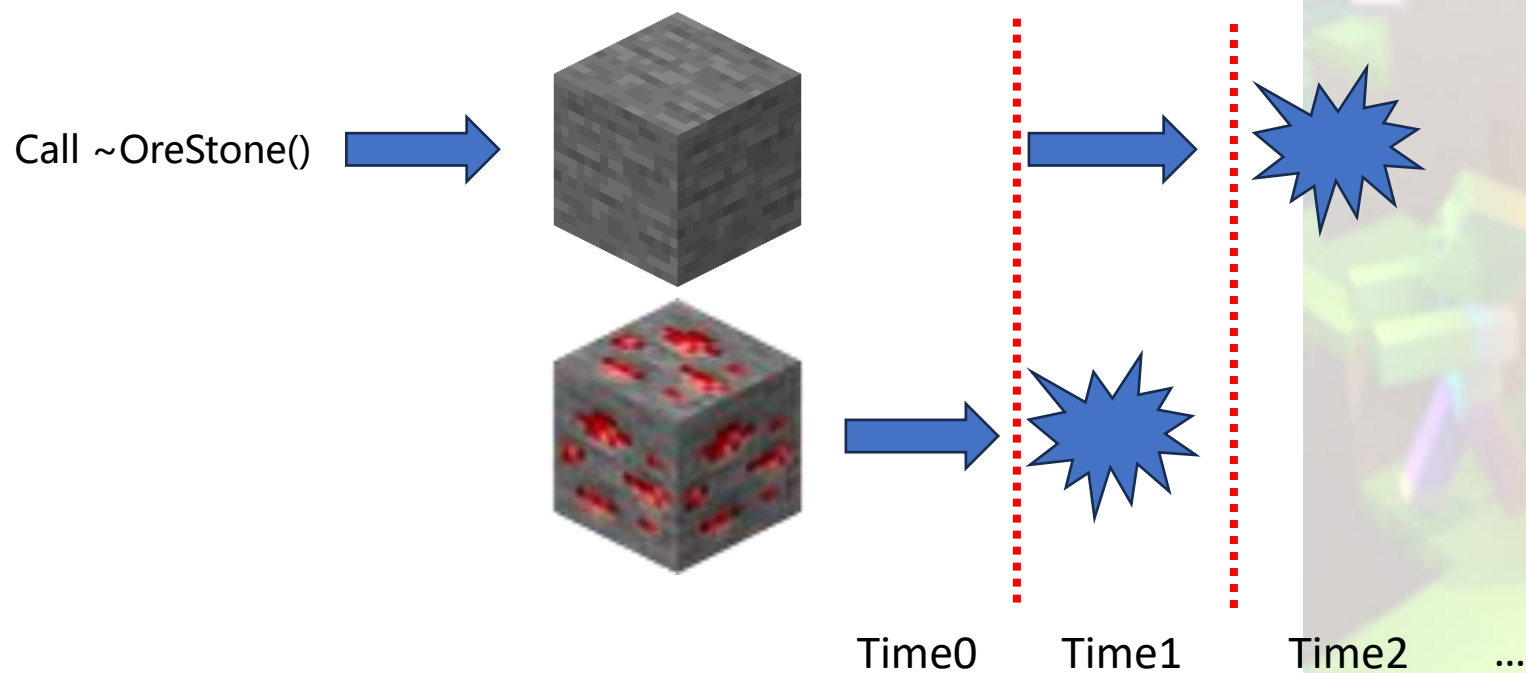
#endif // OOP_REDSTONEORESTONE_H
```



註：「解構鏈」一詞並不一定是正式名稱 (Google 來源)

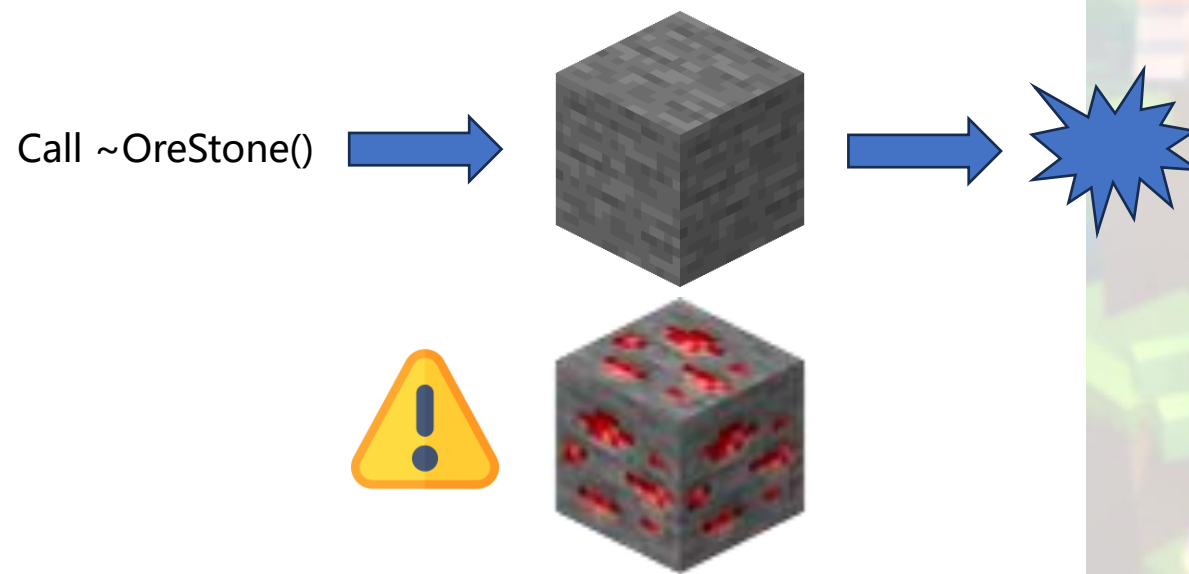
Virtual Destructor

- 考慮到一個具有繼承的物件，我們「理當」能夠得到父類物件所有的函式與成員
- 在釋放物件的情況下，我們正常情況也應當釋放子類成員與父類成員



Virtual Destructor

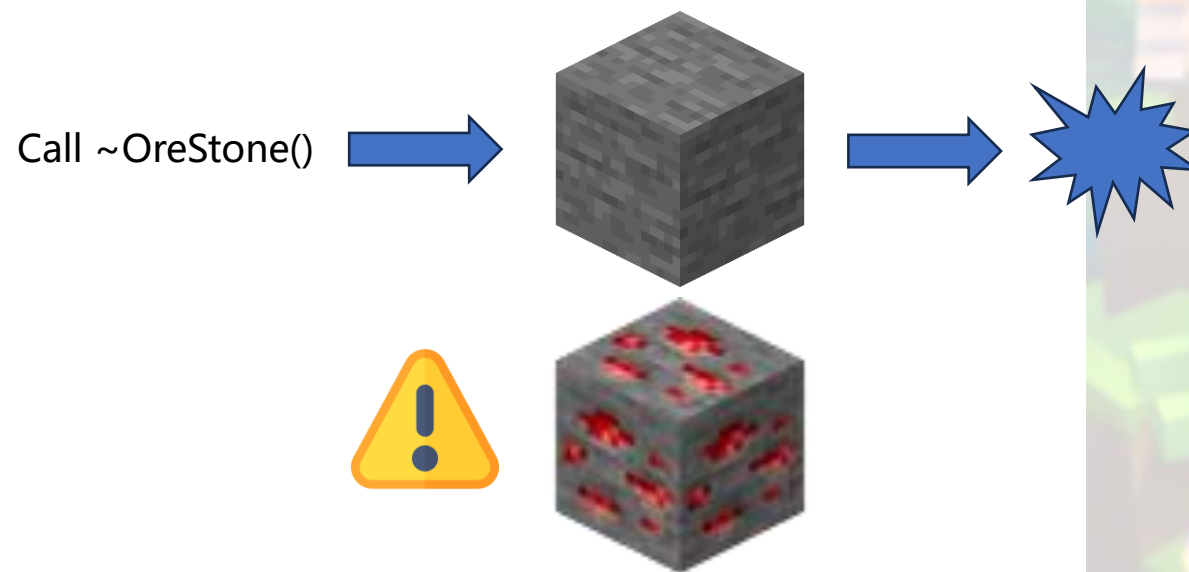
- 解構子須不須要是 Virtual 的?



- 這類的行為在 C++ 被稱為 undefined behavior, 你應該要防止這樣的行為發生
 - 有可能會出現 memory leak (取決於編譯器)

Virtual Destructor

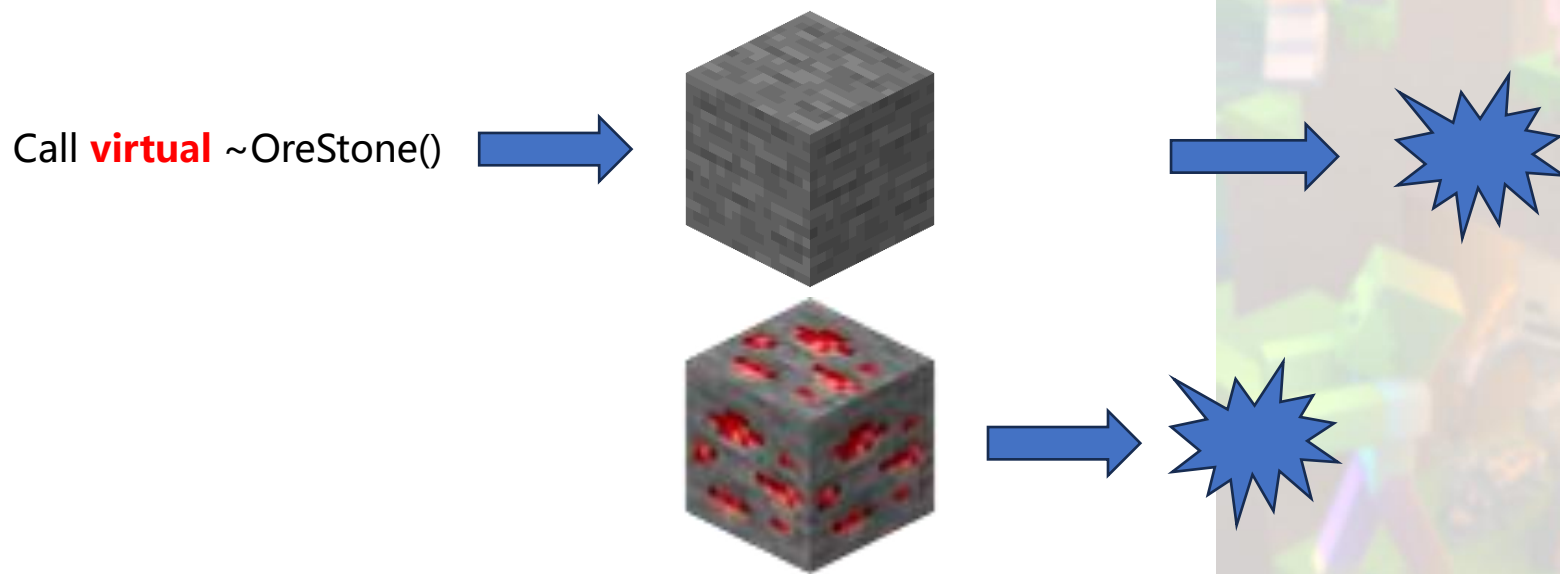
- 為什麼會發生？



- 因為父類的解構子被 Early Binding，導致解構這件事情使用的是父類的解構子
- 父類上層沒有其他解構子了，但子類確實沒有被解構，導致子類的資源沒有完全被釋放

Virtual Destructor

- 假設我們有 Virtual Destructor 的話，由於 Late Binding，所以能夠順利呼叫子類的 Destructor



- 當我們呼叫子類的 Destructor 時，由於「解構鏈」的關係，可以正常向上釋放父類的資源

Virtual Destructor

- 對於一個有繼承的物件，你應該要在父類讓 Destructor 是 Virtual 的，來確保所有資源均能被釋放。

```
#ifndef ORE_STONE_HPP
#define ORE_STONE_HPP

class OreStone {
public:
    std::string GetOre(){
        return "Unknown Ore";
    }

    virtual ~OreStone(){
    }
};

#endif
```

```
#ifndef OOP_REDSTONEORESTONE_H
#define OOP_REDSTONEORESTONE_H

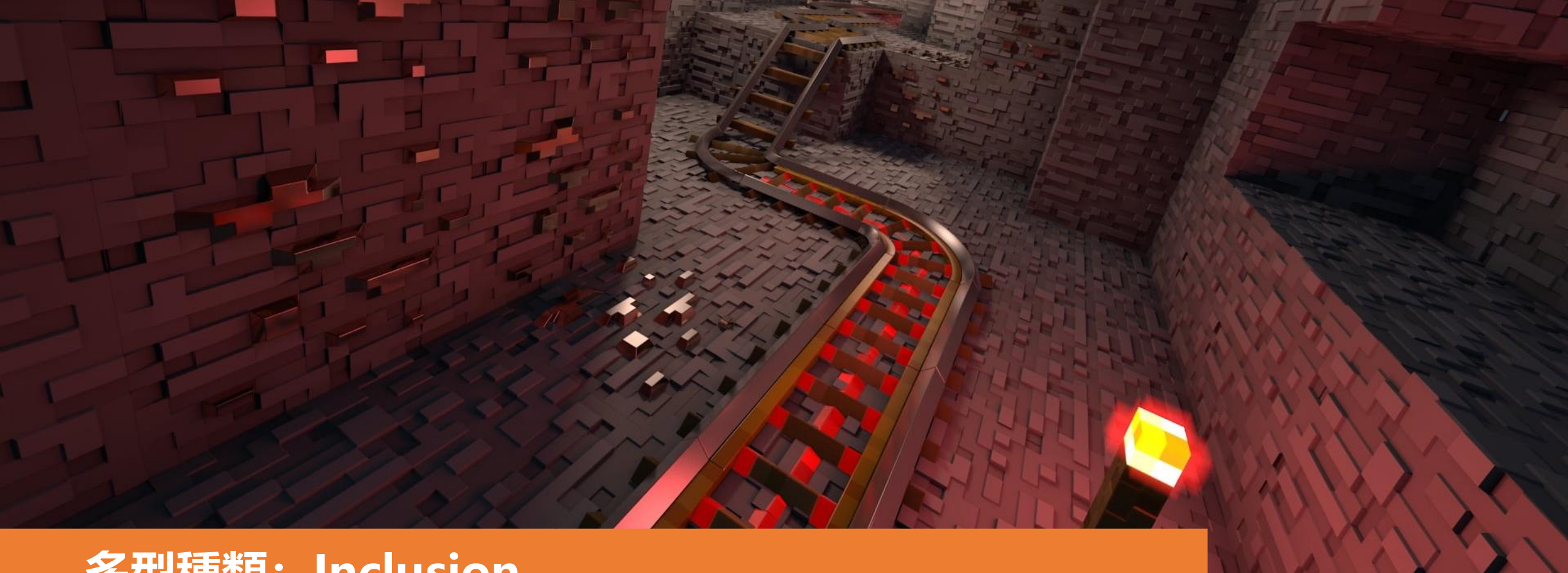
#include "OreStone.hpp"

class RedStoneOreStone : public OreStone {
public:
    std::string GetOre() {
        return "Red Stone";
    }

    ~RedStoneOreStone() override{
    }

    void ShineStoneTexture(){
        /* Do something */
    }
};

#endif // OOP_REDSTONEORESTONE_H
```



多型種類：Inclusion

進入到第一種多型種類：Inclusion（最常見的多型）

多型種類：Inclusion

- 接下來我們希望製作一個佇列熔爐（Queue Furnace），來讓不同的礦物可以逐一燒成錠



金礦 → 金錠
鐵礦 → 鐵錠
...

多型種類: Inclusion

- 因此，我們希望能夠先做出鐵礦與金礦，並且使其繼承一個（可以被燒製的）礦物的類別
- 我們期望 Smelt() 在金礦時會回傳 "Gold Ingot" 且在鐵礦時會回傳 "Iron Ingot"（具有不同的實作細節）。

可以被燒製的礦物

金礦
鐵礦
...

金錠
鐵錠

```
class SmeltableOre {
private:
    /// 燒製的結果
    std::string smeltResult;
public:
    SmeltableOre(std::string result){
        this->smeltResult = result;
    }

    /// 燒製並得到錠
    virtual std::string Smelt(){
        return this->smeltResult;
    }
};
```

```
#include "SmeltableOre.hpp"

class IronOre : public SmeltableOre{
public:
    IronOre() : SmeltableOre( result: "Iron Ingot"){

    }
};
```

```
#include "SmeltableOre.hpp"

class GoldOre : public SmeltableOre {
    GoldOre() : SmeltableOre( result: "Gold Ingot"){

    }
};
```

多型種類：Inclusion

- 接著我們開始造佇列熔爐，造的策略：
 - 用兩個 vector 來維護兩種不同的礦物
 - 用一個 vector 來維護目前要燒哪個礦物
 - 用 setter 來新增礦物
 - 若 vector 目前指向的位置是鐵礦，就燒鐵礦
 - 若 vector 目前指向的位置是金礦，就燒金礦

```
#include <vector>

#include "GoldOre.h"
#include "IronOre.hpp"

class QueueFurnace {
private:
    int goldOreQueueIndex = 0;
    std::vector<GoldOre> goldOreQueue;
    int ironOreQueueIndex = 0;
    std::vector<IronOre> ironOreQueue;
    int index = 0;
    std::vector<std::string> typeOfIndex;
```

```
public:
    QueueFurnace() = default;
    void AddIronOre(IronOre ironOre){
        ironOreQueue.push_back(ironOre);
        typeOfIndex.push_back("Iron Ore");
        ironOreQueueIndex += 1;
    }
    void AddGoldOre(GoldOre goldOre){
        goldOreQueue.push_back(goldOre);
        typeOfIndex.push_back("Gold Ore");
        goldOreQueueIndex += 1;
    }
    std::string SmeltItem(){
        std::string result = "";
        if(index == typeOfIndex.size()){
            return result;
        }
        if(typeOfIndex[index] == "IronOre"){
            result = ironOreQueue[ironOreQueueIndex].Smelt();
            ironOreQueueIndex += 1;
        }else{
            result = goldOreQueue[goldOreQueueIndex].Smelt();
            goldOreQueueIndex += 1;
        }
        index += 1;
        return result;
    }
};
```

這樣有什麼問題？

多型種類: Inclusion

- 永遠 smelt Gold Ore
- QueueIndex 邏輯錯了
- index 0 -> index 1 -> ... -> index n -> index n+1

```
#include <vector>

#include "GoldOre.h"
#include "IronOre.hpp"

class QueueFurnace {
private:
    int goldOreQueueIndex = 0;
    std::vector<GoldOre> goldOreQueue;
    int ironOreQueueIndex = 0;
    std::vector<IronOre> ironOreQueue;
    int index = 0;
    std::vector<std::string> typeOfIndex;
```

```
public:
    QueueFurnace() = default;
    void AddIronOre(IronOre ironOre){
        ironOreQueue.push_back(ironOre);
        typeOfIndex.push_back("Iron Ore");
        ironOreQueueIndex += 1;
    }
    void AddGoldOre(GoldOre goldOre){
        goldOreQueue.push_back(goldOre);
        typeOfIndex.push_back("Gold Ore");
        goldOreQueueIndex += 1;
    }
    std::string SmeltItem(){
        std::string result = "";
        if(index == typeOfIndex.size()){
            return result;
        }
        if(typeOfIndex[index] == "IronOre"){
            result = ironOreQueue[ironOreQueueIndex].Smelt();
            ironOreQueueIndex += 1;
        }else{
            result = goldOreQueue[goldOreQueueIndex].Smelt();
            goldOreQueueIndex += 1;
        }
        index += 1;
        return result;
    }
};
```

這樣有什麼問題?

多型種類: Inclusion

- 造的策略:
 - 用兩個 vector 來維護兩種不同的礦物
 - 用一個 vector 來維護目前要燒哪個礦物
 - 用 setter 來新增礦物
 - 若 vector 目前指向的位置是鐵礦，就燒鐵礦
 - 若 vector 目前指向的位置是金礦，就燒金礦
- 這樣有什麼問題?
 - 如果需要第三種礦物（銅），那麼將會提升這個熔爐的實作複雜度

```
//  
// Created by 黃漢軒 on 2023/11/13.  
//
```

```
#ifndef OOP_QUEUEFURNACE_H  
#define OOP_QUEUEFURNACE_H
```

```
#include <vector>
```

```
#include "GoldOre.h"
```

```
#include "IronOre.hpp"
```

```
#include "CopperOre.h"
```

```
class QueueFurnace {
```

```
private:
```

```
    int goldOreQueueIndex = 0;
```

```
    std::vector<GoldOre> goldOreQueue;
```

```
    int ironOreQueueIndex = 0;
```

```
    std::vector<IronOre> ironOreQueue;
```

```
    int copperOreQueueIndex = 0;
```

```
    std::vector<CopperOre> copperOreQueue;
```

```
    int index = 0;
```

```
    std::vector<std::string> typeOfIndex;
```

```
public:
```

```
    QueueFurnace() = default;
```

```
    void AddIronOre(IronOre ironOre){
```

```
        ironOreQueue.push_back(ironOre);
```

```
        typeOfIndex.push_back("Iron Ore");
```

```
        ironOreQueueIndex += 1;
```

```
    }
```

```
    void AddGoldOre(GoldOre goldOre){
```

```
        goldOreQueue.push_back(goldOre);
```

```
        typeOfIndex.push_back("Gold Ore");
```

```
        goldOreQueueIndex += 1;
```

```
    }
```

```
    void AddCopperOre(CopperOre goldOre){
```

```
        copperOreQueue.push_back(goldOre);
```

```
        typeOfIndex.push_back("Copper Ore");
```

```
        copperOreQueueIndex += 1;
```

```
    }
```

```
    std::string SmeltItem(){
```

```
        std::string result = "";
```

```
        if(index == typeOfIndex.size()){
```

```
            return result;
```

```
        }
```

```
        if(typeOfIndex[index] == "Iron Ore"){
```

```
            result = ironOreQueue[ironOreQueueIndex].Smelt();
```

```
            ironOreQueueIndex += 1;
```

```
        }else if(typeOfIndex[index] == "Gold Ore"){
```

```
            result = goldOreQueue[goldOreQueueIndex].Smelt();
```

```
            goldOreQueueIndex += 1;
```

```
        }else if(typeOfIndex[index] == "Copper Ore"){
```

```
            result = copperOreQueue[copperOreQueueIndex].Smelt();
```

```
            copperOreQueueIndex += 1;
```

```
        }
```

```
        index += 1;
```

```
        return result;
```

```
    }
```

```
};
```

```
#endif // OOP_QUEUEFURNACE_H
```

多型種類: Inclusion

- 這樣有什麼問題?
 - 如果需要第三種礦物（銅），那麼將會提升這個熔爐的實作複雜度
- 深入觀察一下程式，會發現一些有趣的地方：
 1. 基本上，只是因為礦物型態不一樣所以創了兩個容器出來，但是這些礦物可以被 Object Casting 成父類型態，這樣只需要一個容器

```
int goldOreQueueIndex = 0;
std::vector<GoldOre> goldOreQueue;
int ironOreQueueIndex = 0;
std::vector<IronOre> ironOreQueue;
int index = 0;
std::vector<std::string> typeOfIndex;
```

```
void AddIronOre IronOre ironOre {
    ironOreQueue.push_back(ironOre);
    typeOfIndex.push_back("Iron Ore");
    ironOreQueueIndex += 1;
}

void AddGoldOre GoldOre goldOre {
    goldOreQueue.push_back(goldOre);
    typeOfIndex.push_back("Gold Ore");
    goldOreQueueIndex += 1;
}
```

Object casting 後

```
#include <vector>

#include "SmeltableOre.hpp"
#include "GoldOre.h"
#include "IronOre.hpp"

class QueueFurnace {
private:
    int index = 0;
    std::vector<SmeltableOre*> oreQueue;
public:
    QueueFurnace() = default;
    void AddOre(SmeltableOre* ore){
        oreQueue.push_back(ore);
    }

    std::string SmeltItem(){
        std::string result = "";
        if(index == oreQueue.size()){
            return result;
        }
        result = oreQueue[index]->Smelt();
        index += 1;
        return result;
    }
};
```

多型種類: Inclusion

- 這樣有什麼問題?
 - 如果我們今天需要第三種礦物，那麼將會提升這個熔爐的實作複雜度
- 深入觀察一下程式，你會發現一些有趣的地方：
 1. 基本上，只是因為礦物型態不一樣所以創了兩個容器出來，但是這些礦物可以被 Object Casting 成父類型態，這樣只需要一個容器
 2. 基本上，唯一的差別在他們呼叫了不同類別的 Smelt，但我們可以保證礦物都有 Smelt()，似乎可以被 Object Casting 成父類型態後呼叫 Smelt

```
std::string SmeltItem(){
    std::string result = "";
    if(index == typeOfIndex.size()){
        return result;
    }
    if(typeOfIndex[index] == "Iron Ore"){
        result = ironOreQueue[ironOreQueueIndex].Smelt();
        ironOreQueueIndex += 1;
    }else if(typeOfIndex[index] == "Gold Ore"){
        result = goldOreQueue[goldOreQueueIndex].Smelt();
        goldOreQueueIndex += 1;
    }
    index += 1;
    return result;
}
```

Object casting 後

```
private:
    int index = 0;
    std::vector<SmeltableOre*> oreQueue;
public:
    QueueFurnace() = default;
    void AddOre(SmeltableOre* ore){
        oreQueue.push_back(ore)
    }
    std::string SmeltItem(){
        std::string result = "";
        if(index == oreQueue.size()){
            return result;
        }
        result = oreQueue[index]->Smelt();
        index += 1;
        return result;
    }
};
```

多型種類: Inclusion

- 這樣有什麼問題?
 - 如果我們今天需要第三種礦物，那麼將會提升這個熔爐的實作複雜度
- 深入觀察一下程式，你會發現一些有趣的地方：
 1. 基本上我們只是因為礦物型態不一樣所以創了兩個 Queue 出來，但是這些礦物可以被 Object Casting 成父類型態，這樣只需要一個 Queue
 2. 基本上，唯一的差別在他們呼叫了不同類別的 Smelt，但我們可以保證礦物都有 Smelt()，似乎可以被 Object Casting 成父類型態後呼叫 Smelt
 3. 我們可以把容器調整成父類型態，並且取得指標物件後呼叫 Smelt() 並取得子類的 Smelt() 回傳值

```
#include <vector>

#include "SmeltableOre.hpp"
#include "GoldOre.h"
#include "IronOre.hpp"

class QueueFurnace {
private:
    int index = 0;
    std::vector<SmeltableOre*> oreQueue;
public:
    QueueFurnace() = default;
    void AddOre(SmeltableOre* ore){
        oreQueue.push_back(ore);
    }
    std::string SmeltItem(){
        std::string result = "";
        if(index == oreQueue.size()){
            return result;
        }
        result = oreQueue[index]->Smelt();
        index += 1;
        return result;
    }
};
```

多型種類：Inclusion

- 這樣的實作有什麼好處？
 - 不用特判礦物，基本上只要礦物有繼承 SmeltableOre 就能夠丟進來燒（同時也符合語意）
 - 如果不用特判，那我們就能大幅減少因為特判所導致的大量實作細節
- 透過 Inclusion Polymorphism，我們可以大幅減少因為「同樣函數原型、不同類別、不同實作細節」所造成的特判情況
 - 利用父類別的型態、接收不同子類別的物件、做相同的動作、引發不同的行為^[1]

```
#include "GoldOre.h"
#include "IronOre.hpp"

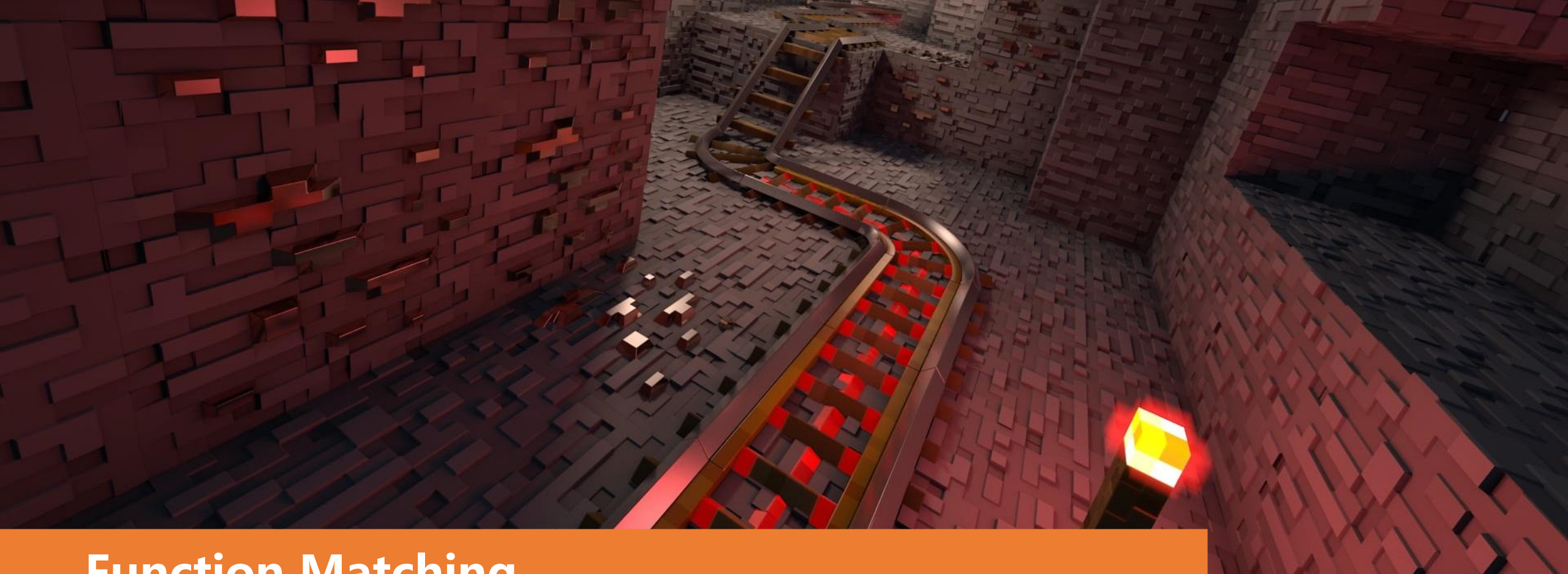
class QueueFurnace {
private:
    int goldOreQueueIndex = 0;
    std::vector<GoldOre> goldOreQueue;
    int ironOreQueueIndex = 0;
    std::vector<IronOre> ironOreQueue;
    int index = 0;
    std::vector<std::string> typeOfIndex;
public:
    QueueFurnace() = default;
    void AddIronOre(IronOre ironOre){
        ironOreQueue.push_back(ironOre);
        typeOfIndex.push_back("Iron Ore");
        ironOreQueueIndex += 1;
    }
    void AddGoldOre(GoldOre goldOre){
        goldOreQueue.push_back(goldOre);
        typeOfIndex.push_back("Gold Ore");
        goldOreQueueIndex += 1;
    }
    std::string SmeltItem(){
        std::string result = "";
        if(index == typeOfIndex.size()){
            return result;
        }
        if(typeOfIndex[index] == "IronOre"){
            result = ironOreQueue[ironOreQueueIndex].Smelt();
            ironOreQueueIndex += 1;
        }else{
            result = goldOreQueue[goldOreQueueIndex].Smelt();
            goldOreQueueIndex += 1;
        }
        index += 1;
        return result;
    }
};
```

oreQueue 裡存的是「父類別指標」，但指向的是「子類別物件」

```
#include <vector>

#include "SmeltableOre.hpp"
#include "GoldOre.h"
#include "IronOre.hpp"

class QueueFurnace {
private:
    int index = 0;
    std::vector<SmeltableOre*> oreQueue;
public:
    QueueFurnace() = default;
    void AddOre(SmeltableOre* ore){
        oreQueue.push_back(ore);
    }
    std::string SmeltItem(){
        std::string result = "";
        if(index == oreQueue.size()){
            return result;
        }
        result = oreQueue[index]->Smelt();
        index += 1;
        return result;
    }
};
```

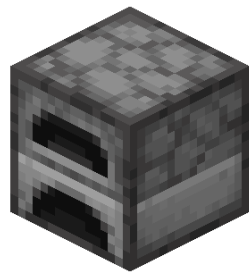


Function Matching

在進入到第二個多型種類之前，我們想先談一下 C++ 針對於同名但不同傳入參數的行為

Function Matching

- 回來看一下熔爐的實作



- 你會發現由於我們有不同的礦物，所以我們將這個實作細節分開實作
 - 當我們想要新增鐵礦進入佇列熔爐時，我們會呼叫 `AddIronOre()`
 - 當我們想要新增金礦進入佇列熔爐時，我們會呼叫 `AddGoldOre()`

```
class QueueFurnace {
private:
    int goldOreQueueIndex = 0;
    std::vector<GoldOre> goldOreQueue;
    int ironOreQueueIndex = 0;
    std::vector<IronOre> ironOreQueue;
    int index = 0;
    std::vector<std::string> typeOfIndex;
public:
    QueueFurnace() = default;
    void AddIronOre(IronOre ironOre){
        ironOreQueue.push_back(ironOre);
        typeOfIndex.push_back("Iron Ore");
        ironOreQueueIndex += 1;
    }
    void AddGoldOre(GoldOre goldOre){
        goldOreQueue.push_back(goldOre);
        typeOfIndex.push_back("Gold Ore");
        goldOreQueueIndex += 1;
    }
    std::string SmeltItem(){
        std::string result = "";
        if(index == typeOfIndex.size()){
            return result;
        }
        if(typeOfIndex[index] == "IronOre"){
            result = ironOreQueue[ironOreQueueIndex].Smelt();
            ironOreQueueIndex += 1;
        }else{
            result = goldOreQueue[goldOreQueueIndex].Smelt();
            goldOreQueueIndex += 1;
        }
        index += 1;
        return result;
    }
};
```

Function Matching

- 如果我們今天把名稱統一改成 AddOre()?
 - <https://onlinegdb.com/7iBKlYWCM>

```
class QueueFurnace {
private:
    int goldOreQueueIndex = 0;
    std::vector<GoldOre> goldOreQueue;
    int ironOreQueueIndex = 0;
    std::vector<IronOre> ironOreQueue;
    int index = 0;
    std::vector<std::string> typeOfIndex;
public:
    QueueFurnace() = default;
    void AddOre(IronOre ironOre){
        ironOreQueue.push_back(ironOre);
        typeOfIndex.push_back("Iron Ore");
        ironOreQueueIndex += 1;
    }
    void AddOre(GoldOre goldOre){
        goldOreQueue.push_back(goldOre);
        typeOfIndex.push_back("Gold Ore");
        goldOreQueueIndex += 1;
    }
    std::string SmeltItem(){
        std::string result = "";
        if(index == typeOfIndex.size()){
            return result;
        }
        if(typeOfIndex[index] == "Iron Ore"){
            result = ironOreQueue[ironOreQueueIndex].Smelt();
            ironOreQueueIndex += 1;
        }else if(typeOfIndex[index] == "Gold Ore"){
            result = goldOreQueue[goldOreQueueIndex].Smelt();
            goldOreQueueIndex += 1;
        }
        index += 1;
        return result;
    }
};
```

Function Matching

- 如果我們今天把名稱統一改成 AddOre()?
- 事實上這是可以運作的
- C++ 會尋找最適合的 Function 去使用它

```
#include "BadPractice_QueueFurnace.h"
#include "GoldOre.h"

int main(){
    QueueFurnace queueFurnace;
    GoldOre goldOre;

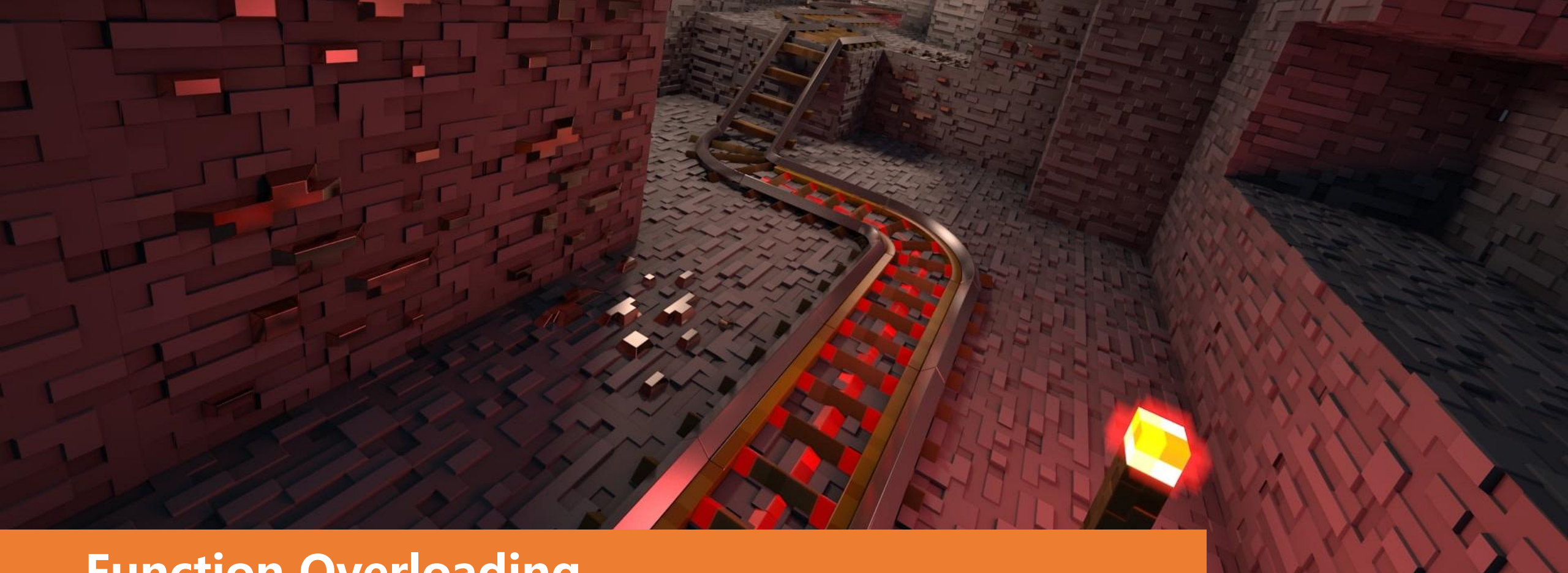
    queueFurnace.AddOre(goldOre);
}
```

```
class QueueFurnace {
private:
    int goldOreQueueIndex = 0;
    std::vector<GoldOre> goldOreQueue;
    int ironOreQueueIndex = 0;
    std::vector<IronOre> ironOreQueue;
    int index = 0;
    std::vector<std::string> typeOfIndex;
public:
    QueueFurnace() = default;
    void AddOre(IronOre ironOre){
        ironOreQueue.push_back(ironOre);
        typeOfIndex.push_back("Iron Ore");
        ironOreQueueIndex += 1;
    }
    void AddOre(GoldOre goldOre){
        goldOreQueue.push_back(goldOre);
        typeOfIndex.push_back("Gold Ore");
        goldOreQueueIndex += 1;
    }
    std::string SmeltItem(){
        std::string result = "";
        if(index == typeOfIndex.size()){
            return result;
        }
        if(typeOfIndex[index] == "Iron Ore"){
            result = ironOreQueue[ironOreQueueIndex].Smelt();
            ironOreQueueIndex += 1;
        }else if(typeOfIndex[index] == "Gold Ore"){
            result = goldOreQueue[goldOreQueueIndex].Smelt();
            goldOreQueueIndex += 1;
        }
        index += 1;
        return result;
    }
};
```

Function Matching

- 事實上，在 C++ 中，Function Name 與 Function 並不是一對一，而是一對多
- 例如我們可以建立許多個同名但不同參數的 Function，來在呼叫時讓 C++ 自動選擇對應的 Function
 - <https://onlinegdb.com/qTRQvkVZGH>
- 透過這個概念，我們可以開始考慮這樣機制為什麼能夠被稱為「多型」





Function Overloading

進入到第二種多型種類：Ad-Hoc – Function Overloading

Function Overloading

- 讓我們考慮一下這個案例：



- Minecraft 有很多個方塊，你可以用手以及用鎬子去破壞方塊
 - 如果你用手去破壞方塊，手沒有所謂的耐久度（事實上手會斷掉）
 - 如果你用鎬子去破壞方塊，鎬子有所謂的耐久度，會扣減

Function Overloading

- Minecraft 有很多個方塊，你可以用手以及用鎬子去破壞方塊
 - 如果你用手去破壞方塊，手沒有所謂的耐久度（事實上手會斷掉）
 - 如果你用鎬子去破壞方塊，鎬子有所謂的耐久度，會扣減
- 假設我們有一個 Function 叫做 Mine()

```
void mine(Block block){  
    /* 用手挖方塊 */  
    destroyBlock(block);  
}  
  
void mine(Block block, Tool* tool){  
    /* 用工具挖方塊 */  
    tool->DecreaseDurability();  
    destroyBlock(block);  
}
```

- 一般的 Mine(Block block)
 - 用手去挖方塊
- 有工具的 Mine(Block block, Tool* tool)
 - 先扣道具耐久度
 - 再用工具去挖方塊



Function Overloading

- 我們對於這種相同名稱，不同參數的 Function，稱為 Function Overloading
- 它是一個實作技巧，可以負責處理同名 Function 但不同參數的情況
- 且他可以藉由丟入不同的參數來執行對應的實作細節
- 例如：當我丟入一個工具時，他會減少工具的耐久度，並且將方塊拆除
- 例如：當我不丟入工具時，他會直接將方塊拆除，而不做減少工具耐久度的操作

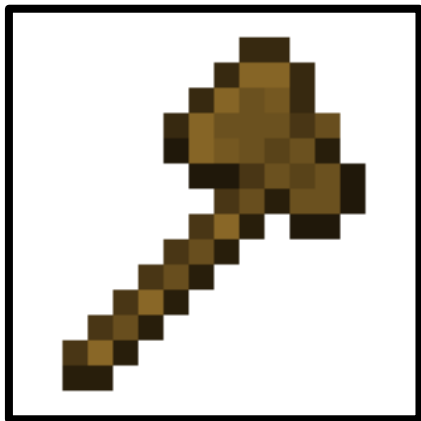
```
int main(){  
    mine( block: Block( name: "紅石磚"));  
}
```

```
void mine(Block block){  
    /* 用手挖方塊 */  
    destroyBlock(block);  
}
```

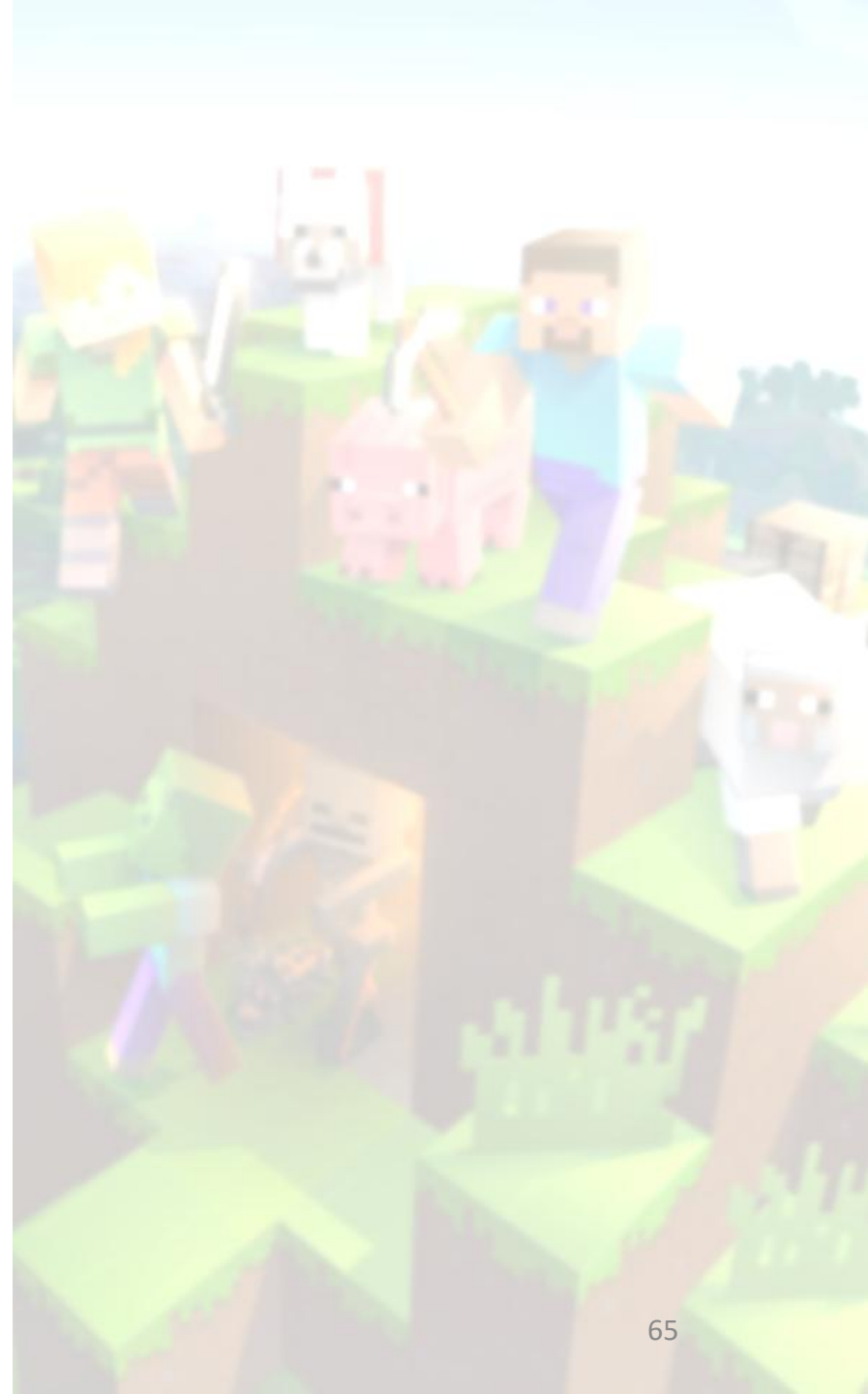
```
void mine(Block block, Tool* tool){  
    /* 用工具挖方塊 */  
    tool->DecreaseDurability();  
    destroyBlock(block);  
}
```

Function Overloading

- 讓我們再考慮一下這個案例：



- Minecraft 有很多種斧頭，每種斧頭都會有不同的挖掘速度：
 - 木斧的砍樹速度很慢，我們給他一個速度值 10
 - 鑽石斧的砍樹速度超級快，我們給他一個速度值 100



Function Overloading

- Minecraft 有很多種斧頭，每種斧頭都會有不同的挖掘速度：
 - 例如木斧的砍樹速度很慢，我們給他一個速度值 10
 - 然而鑽石斧的砍樹速度超級快，我們給他一個速度值 100
- 假設我們有一個 Function 叫做 CropBlock() 並且傳入不同的斧頭：

```
void cropBlock(Block block, WoodAxe axe){  
    /* 用木頭斧砍方塊 */  
}  
  
void cropBlock(Block block, DiamondAxe axe){  
    /* 用鑽石斧砍方塊 */  
}
```

當我們傳入對應的斧頭時，我們就能夠透過
Function Matching 來找到最適合的 Function

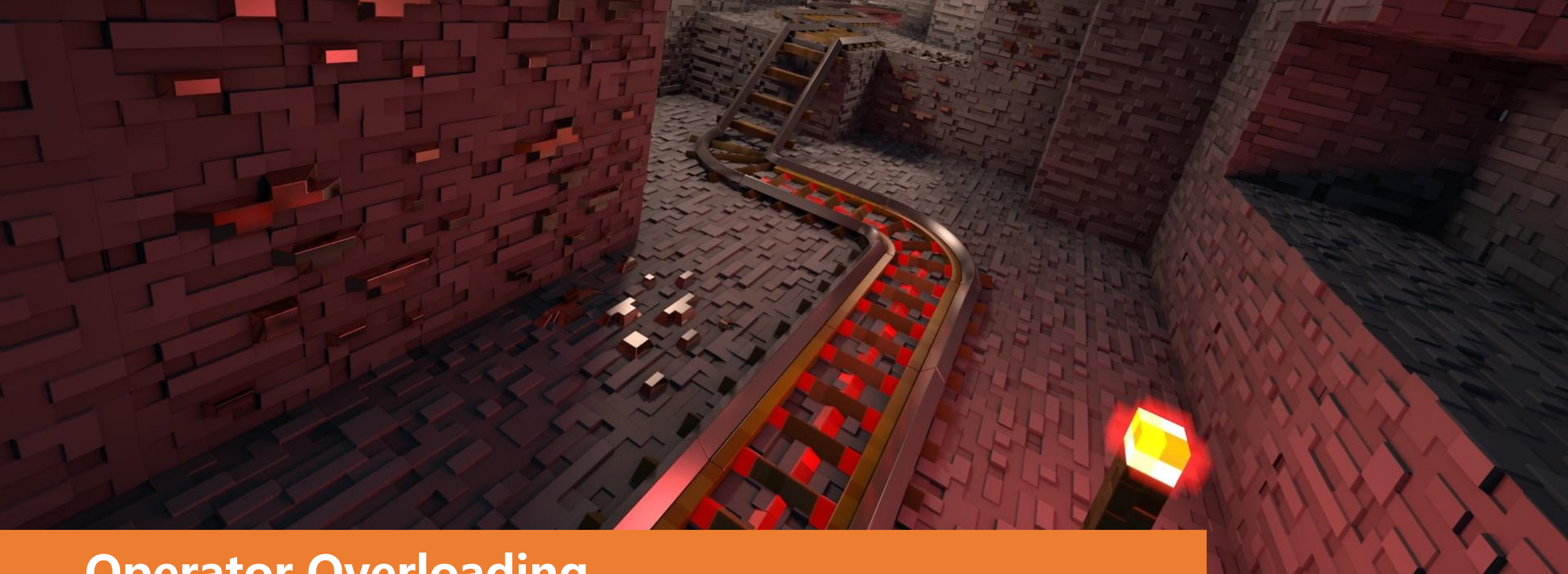
- 如果傳入木斧頭 CropBlock(Block block, WoodAxe axe)
 - 就用速度 10 來砍木頭
- 如果傳入鑽石斧頭 CropBlock(Block block, DiamondAxe axe)
 - 就用速度 100 來砍木頭

比較與討論

- 對於上述提出來的兩種 Function Mapping 的案例，其實有些許差別
 - cropBlock():
 - 使用 Function Overloading 的原因是「**型態**」的不同
 - 希望可以根據傳入的型態來決定使用哪個物件處理方塊
 - mine():
 - 使用 Function Overloading 的原因是「**傳入參數**」的不同
 - 希望能夠透過多載來處理傳入參數的不同來對應到不同的實作細節
- 透過 Function Overloading, 我們可以實踐這些事情
- 但事實上 Function Overloading 對於目前較新穎的語言已經棄用?

```
void cropBlock(Block block, WoodAxe axe){  
    /* 用木頭斧砍方塊 */  
}  
  
void cropBlock(Block block, DiamondAxe axe){  
    /* 用鑽石斧砍方塊 */  
}
```

```
void mine(Block block){  
    /* 用手挖方塊 */  
    destroyBlock(block);  
}  
  
void mine(Block block, Tool* tool){  
    /* 用工具挖方塊 */  
    tool->DecreaseDurability();  
    destroyBlock(block);  
}
```



Operator Overloading

進入到第二種多型種類：Ad-Hoc – Operator Overloading

Operator Overloading

- 設計玩家的移動
 - 利用一個 Location 的類別來記錄玩家的三維座標



```
1 class Location {  
2     private:  
3         int x;  
4         int y;  
5         int z;  
6  
7     public:  
8         Location(int x, int y, int z){  
9             this->x = x;  
10            this->y = y;  
11            this->z = z;  
12        }  
13  
14        int GetX{ return x; }  
15        int GetY{ return y; }  
16        int GetZ{ return z; }  
17    };
```

Operator Overloading

- 設計玩家的移動
 - 利用一個 `Location` 的類別來記錄玩家的三維座標
 - 讓 `Location` 與 `Location` 能夠相加 (或其他運算)
 - 來做出類似位移等相關的操作

```
1 int main()
2 {
3     Location location = Location(5, 6, 7);
4     Location anotherLocation = Location(4, 3, 2);
5
6     Location result = Location(
7         location.GetX() + anotherLocation.GetX(),
8         location.GetY() + anotherLocation.GetY(),
9         location.GetZ() + anotherLocation.GetZ()
10    );
11
12    std::cout << result.GetX() << " "
13              << result.GetY() << " "
14              << result.GetZ() << std::endl;
15 }
```

Operator Overloading

- 如果程式碼可以實作像下圖：
 - 可以讓實作的部分變得更整潔
 - 也更直觀

```
1 int main()
2 {
3     Location location = Location(5, 6, 7);
4     Location anotherLocation = Location(4, 3, 2);
5     Location result = location + anotherLocation;
6
7     std::cout << result.GetX() << " "
8               << result.GetY() << " "
9               << result.GetZ() << std::endl;
10 }
11
```

期望

可讀性

```
1 int main()
2 {
3     Location location = Location(5, 6, 7);
4     Location anotherLocation = Location(4, 3, 2);
5
6     Location result = Location(
7         location.GetX() + anotherLocation.GetX(),
8         location.GetY() + anotherLocation.GetY(),
9         location.GetZ() + anotherLocation.GetZ()
10    );
11
12     std::cout << result.GetX() << " "
13               << result.GetY() << " "
14               << result.GetZ() << std::endl;
15 }
```

原本

Operator Overloading

- 要如何做到這樣的設計？

```
6 Location result = location + anotherLocation;
```

- 如右圖，透過 **overloading** 類別中的 **+** 號，來達到對另一個 Location 進行相加
 - 我們稱這個行為叫做 **Operator Overloading**

```
class Location {  
private:  
    int x;  
    int y;  
    int z;  
  
public:  
    Location(int x, int y, int z) {  
        this->x = x;  
        this->y = y;  
        this->z = z;  
    }  
  
    int GetX() { return x; }  
  
    int GetY() { return y; }  
  
    int GetZ() { return z; }  
  
    Location operator+(Location &another) {  
        return Location(  
            x: x + another.x,  
            y: y + another.y,  
            z: z + another.z  
        );  
    }  
};
```

語法解析

```
Location operator+(Location &another) {  
    return Location(  
        x: x + another.x,  
        y: y + another.y,  
        z: z + another.z  
    );  
}
```

- 拆成三部分來理解

Location

回傳類別（這個函式會回傳一個 Location 物件）

operator+

告訴 C++ 編譯器，這是在重新定義 + 運算子

(Location &another)

傳入參數 → 另一個要跟 this 相加的 Location 物件

Operator Overloading

- 如果內建了一個 Vector 類別，並希望 Location 可以支援這個內建的 Vector 類別進行「運算」

```
1 int main()
2 {
3     Location location = Location(5, 6, 7);
4     Vector shift = Vector(-1, -3, -5);
5
6     Location result = location + shift;
7
8     std::cout << result.GetX() << " "
9               << result.GetY() << " "
10              << result.GetZ() << std::endl;
11 }
```



Operator Overloading

- 如果內建了一個 Vector 類別，並希望 Location 可以支援這個內建的 Vector 類別進行「運算」

```
1 int main()
2 {
3     Location location = Location(5, 6, 7);
4     Vector shift = Vector(-1, -3, -5);
5
6     Location result = location + shift;
7
8     std::cout << result.GetX() << " "
9               << result.GetY() << " "
10              << result.GetZ() << std::endl;
11 }
```

- 這時候我們可以再額外新增一個 Overloading 來達成 Function Overloading，讓 C++ 找到適合的 Function 來進行相加

```
1 class Location {
2     private:
3         int x;
4         int y;
5         int z;
6
7     public:
8         Location(int x, int y, int z){
9             this->x = x;
10            this->y = y;
11            this->z = z;
12        }
13
14        int GetX{ return x; }
15        int GetY{ return y; }
16        int GetZ{ return z; }
17
18        Location operator+(Location &another){
19            return Location(x + another.x,
20                           y + another.y,
21                           z + another.z);
22        }
23
24        Location operator+(Vector &another){
25            return Location(x + another.GetX(),
26                           y + another.GetY(),
27                           z + another.GetZ());
28        }
29    };
30
31 }
```

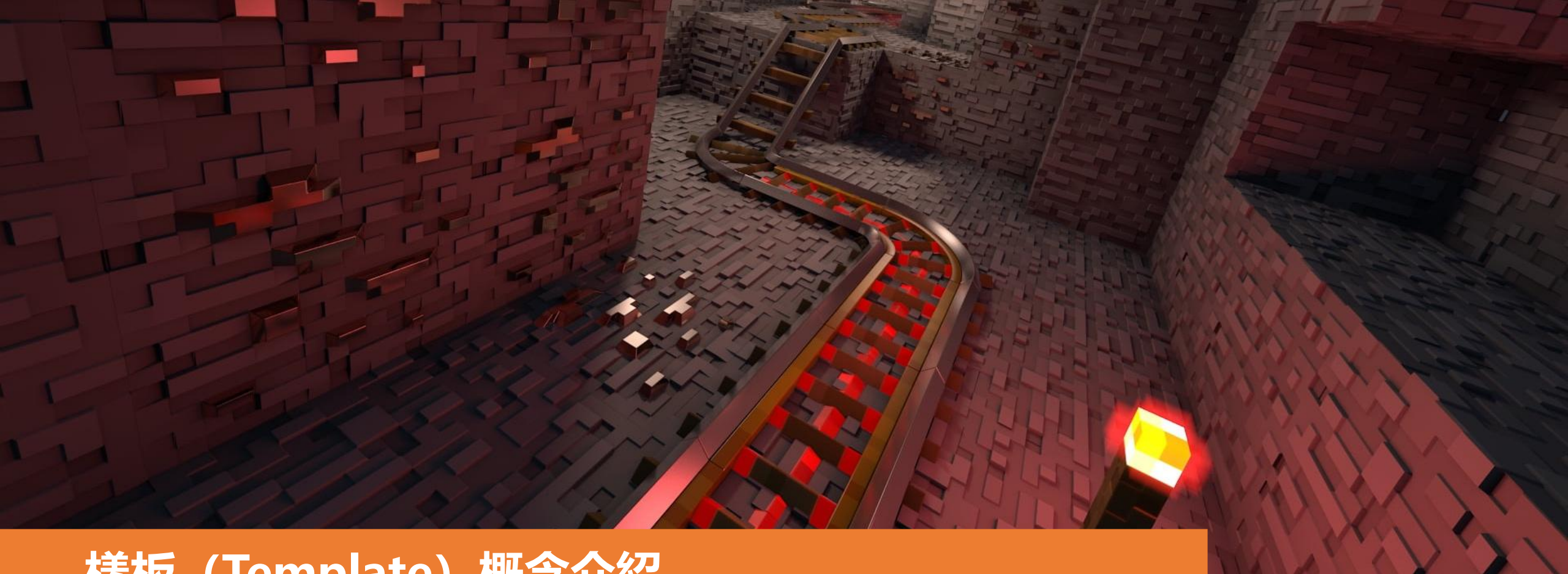
小討論

- 透過 Operator Overloading, 我們可以多載 C++ 內建的運算子, 讓我們可以使物件達成相加、相減等運算
- 對於 C++ 而言, 不是每一個 Operator 都能多載
 - 請參考 [Microsoft 運算子多載](#)
- 另外, 對於一些語言 (例如: Java)
 - Java 本身不支援 operator overloading (如 C++ 那樣的語法)
 - 不支援 Location A + Location B
 - 不支援 ==
 - 但 Java 支援 function overloading
 - 因此可以透過覆寫某些方法 (例如 equals()) 來達成相似的功能



小討論 2

特性	Overloading (多載)	Override (覆寫)
發生位置	同一個類別中	必須在繼承關係中
函式名稱	一樣	一樣
參數 signature	不同	完全相同
virtual 是否需要	不需要	需要
作用	讓函式能處理不同輸入	子類改寫父類行為
時間點	在編譯時決定 (靜態多型)	在執行時決定 (動態多型)

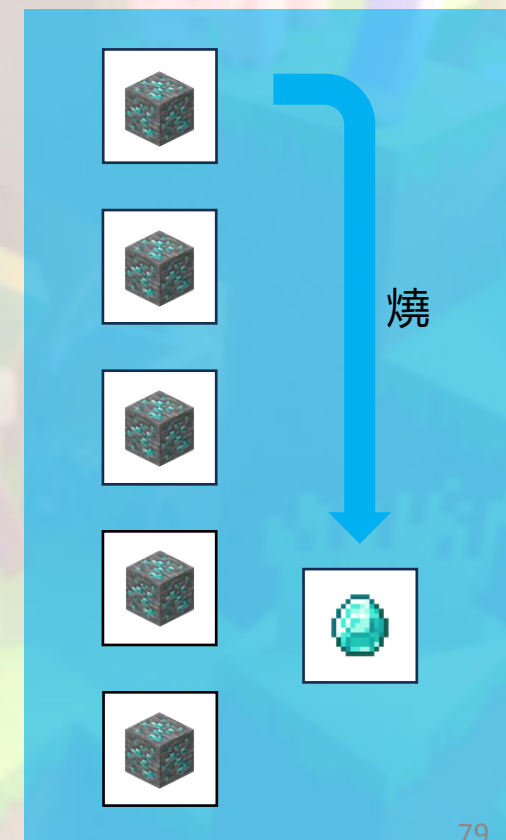
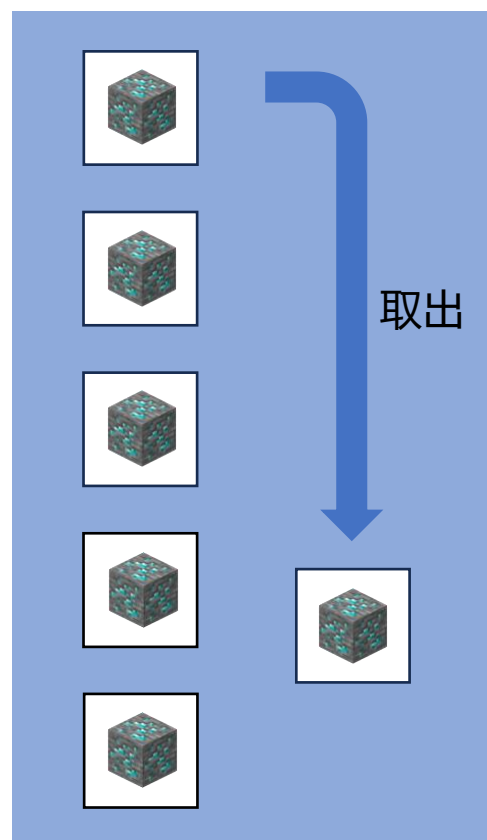
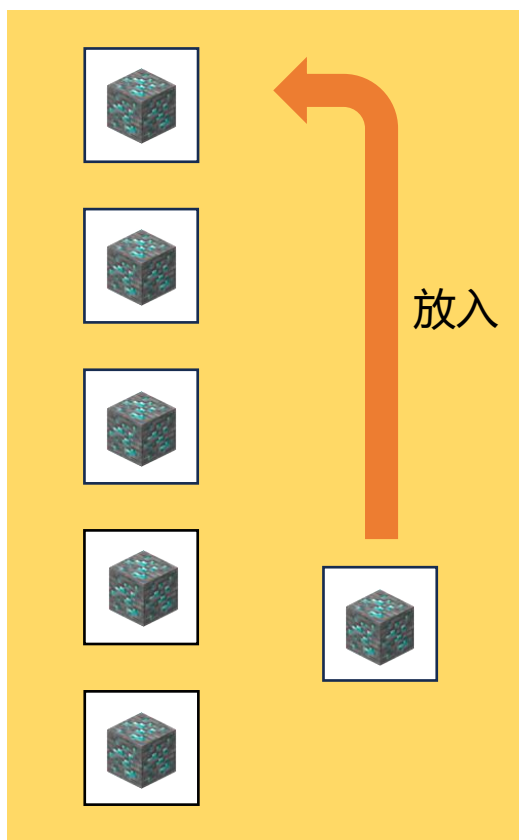


樣板 (Template) 概念介紹

透過前面兩個多型相關的概念後，我們將介紹第三種多型：Parametric

樣板 (Template) 概念介紹

- 我們考慮到因為遊戲的一些機制，我們希望可以在 Minecraft 裡面**自製**一個 Stack，能夠放入某種特定礦石
- 我們可以將礦石放入 Stack、從 Stack 最上層抽取礦石、計算 Stack 有多少個礦石、以及可以將最上層的礦石燒成礦物



樣板 (Template) 概念介紹

- 對於這樣的功能：
 - 我們可以用右邊的程式碼進行實作一個專屬鐵礦的 Stack

```
1 class IronStack{
2     private:
3         std::vector<IronOre> vec;
4
5     public:
6         void push(IronOre ore){
7             vec.push_back(ore);
8         }
9
10        void pop(){
11            vec.pop_back();
12        }
13
14        int size(){
15            return vec.size();
16        }
17
18        std::string smelt(){
19            IronOre ore = vec.back();
20            vec.pop_back();
21            return ore.Smelt();
22        }
23 };
```

樣板 (Template) 概念介紹

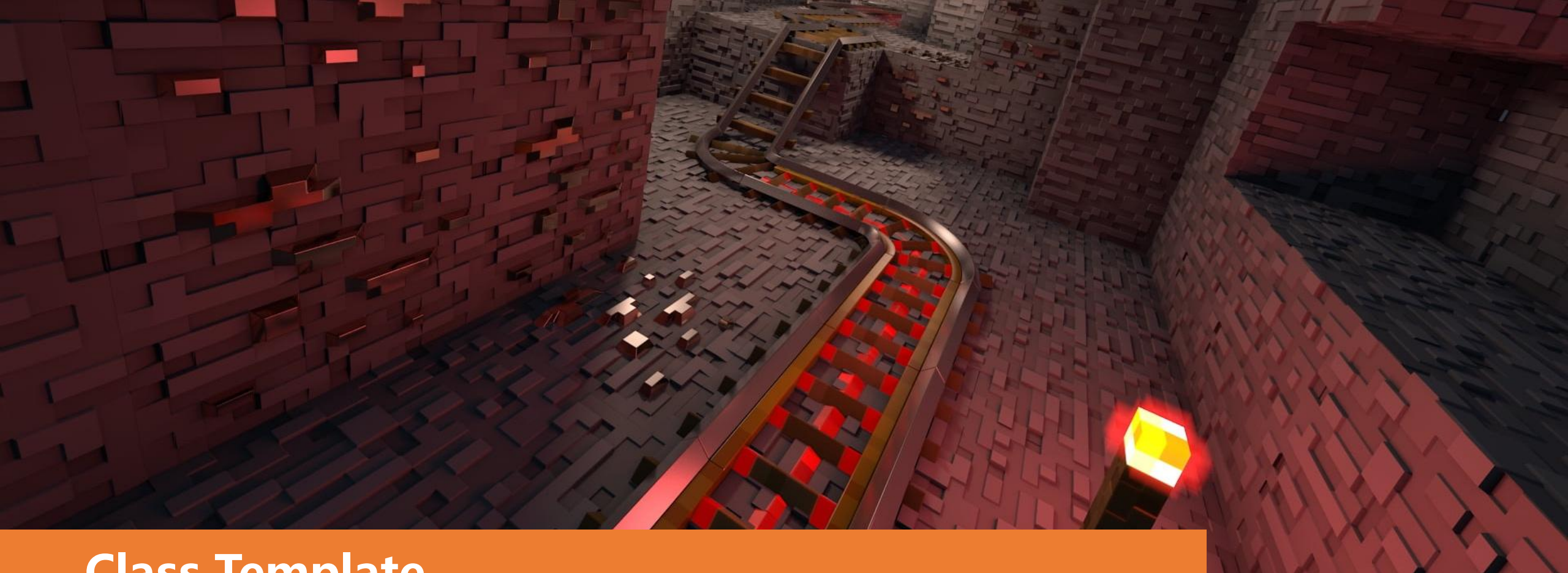
- 對於這樣的功能：
 - 我們可以用右邊的程式碼進行實作一個專屬鐵礦的 Stack
 - 我們還能用右邊的程式碼進行實作一個專屬金礦的 Stack
 - ...

```
1 class IronStack{
2     private:
3         std::vector<GoldOre> vec;
4
5     public:
6         void push(GoldOre ore){
7             vec.push_back(ore);
8         }
9
10        void pop(){
11            vec.pop_back();
12        }
13
14        int size(){
15            return vec.size();
16        }
17
18        std::string smelt(){
19            GoldOre ore = vec.back();
20            vec.pop_back();
21            return ore.Smelt();
22        }
23    };
24 }
```

樣板 (Template) 概念介紹

- 對於這樣的功能：
 - 我們可以用右邊的程式碼進行實作一個專屬鐵礦的 Stack
 - 我們還能用右邊的程式碼進行實作一個專屬金礦的 Stack
 - ...
- 這樣有什麼問題？
 - 如果我們要支援更多的礦物，我們就要創更多的類別出來
 - 如果我們有 100 種礦物就要創 100 個，維護性低
- 要怎麼解決這個問題？

```
1 class IronStack{
2     private:
3         std::vector<GoldOre> vec;
4
5     public:
6         void push(GoldOre ore){
7             vec.push_back(ore);
8         }
9
10        void pop(){
11            vec.pop_back();
12        }
13
14        int size(){
15            return vec.size();
16        }
17
18        std::string smelt(){
19            GoldOre ore = vec.back();
20            vec.pop_back();
21            return ore.Smelt();
22        }
23    };
```



Class Template

- 我們將解釋 Class 如何使用樣板的概念來達成 Parametric 多型

Class Template

- 回顧這個問題，我們可以觀察一下金礦 Stack 與鐵礦 Stack 之間的差異
 - 主要差在**類別名稱與型態**

```
#ifndef OOP_IRONSTACK_H
#define OOP_IRONSTACK_H

#include "IronOre.hpp"

class IronStack {
private:
    std::vector<IronOre> vec;
public:
    void push(IronOre ore){
        vec.push_back(ore);
    }
    void pop(){
        vec.pop_back();
    }
    int size(){
        return vec.size();
    }
    std::string smelt(){
        IronOre ore = vec.back();
        vec.pop_back();
        return ore.Smelt();
    }
};

#endif // OOP_IRONSTACK_H
```

```
#ifndef OOP_GOLDSTACK_H
#define OOP_GOLDSTACK_H

#include "GoldOre.h"

class GoldStack {
private:
    std::vector<GoldOre> vec;
public:
    void push(GoldOre ore){
        vec.push_back(ore);
    }
    void pop(){
        vec.pop_back();
    }
    int size(){
        return vec.size();
    }
    std::string smelt(){
        GoldOre ore = vec.back();
        vec.pop_back();
        return ore.Smelt();
    }
};

#endif // OOP_GOLDSTACK_H
```

Class Template

- 假設我們能夠傳入這些型態，似乎我們就能夠併成同一個類別？

```
#ifndef OOP_IRONSTACK_H
#define OOP_IRONSTACK_H

#include "IronOre.hpp"

class IronStack {
private:
    std::vector<IronOre> vec;
public:
    void push(IronOre ore){
        vec.push_back(ore);
    }
    void pop(){
        vec.pop_back();
    }
    int size(){
        return vec.size();
    }
    std::string smelt(){
        IronOre ore = vec.back();
        vec.pop_back();
        return ore.Smelt();
    }
};

#endif // OOP_IRONSTACK_H
```

```
#ifndef OOP_GOLDSTACK_H
#define OOP_GOLDSTACK_H

#include "GoldOre.h"

class GoldStack {
private:
    std::vector<GoldOre> vec;
public:
    void push(GoldOre ore){
        vec.push_back(ore);
    }
    void pop(){
        vec.pop_back();
    }
    int size(){
        return vec.size();
    }
    std::string smelt(){
        GoldOre ore = vec.back();
        vec.pop_back();
        return ore.Smelt();
    }
};

#endif // OOP_GOLDSTACK_H
```

模板化

```
#ifndef OOP_STACK_H
#define OOP_STACK_H

#include <vector>
#include <string>

class Stack {
private:
    std::vector<T> vec;
public:
    void push(T ore){
        vec.push_back(ore);
    }
    void pop(){
        vec.pop_back();
    }
    int size(){
        return vec.size();
    }
    std::string smelt(){
        T ore = vec.back();
        vec.pop_back();
        return ore.Smelt();
    }
};

#endif // OOP_STACK_H 85
```

Class Template

- 我們可以使用 Template 來讓類別使用通用型別
 - 未來將類別中通用型別直接取代成「指定」的型別
- 在類別中的 T 我們統稱叫做「通用類別 (Generic)」
 - 對於這個 T 而言，將會被使用者所傳入的型別所取代

```
#ifndef OOP_STACK_H
#define OOP_STACK_H

#include <vector>
#include <string>

template <typename T>
class Stack {
private:
    std::vector<T> vec;
public:
    void push(T ore){
        vec.push_back(ore);
    }
    void pop(){
        vec.pop_back();
    }
    int size(){
        return vec.size();
    }
    std::string smelt(){
        T ore = vec.back();
        vec.pop_back();
        return ore.Smelt();
    }
};

#endif // OOP_STACK_H
```

Class Template

- 概念圖

```
#include <iostream>

#include "../include/IronOre.hpp"
#include "../include/Stack.h"

int main(){
    Stack<IronOre> stack;

    stack.push( ore: IronOre());
    std::cout << stack.smelt() << std::endl;
}
```

+

```
#ifndef OOP_STACK_H
#define OOP_STACK_H

#include <vector>
#include <string>

template <typename T>
class Stack {
private:
    std::vector<T> vec;
public:
    void push(T ore){
        vec.push_back(ore);
    }
    void pop(){
        vec.pop_back();
    }
    int size(){
        return vec.size();
    }
    std::string smelt(){
        T ore = vec.back();
        vec.pop_back();
        return ore.Smelt();
    }
};

#endif // OOP_STACK_H
```

=

```
#ifndef OOP_IRONSTACK_H
#define OOP_IRONSTACK_H

#include "IronOre.hpp"

class IronStack {
private:
    std::vector<IronOre> vec;
public:
    void push(IronOre ore){
        vec.push_back(ore);
    }
    void pop(){
        vec.pop_back();
    }
    int size(){
        return vec.size();
    }
    std::string smelt(){
        IronOre ore = vec.back();
        vec.pop_back();
        return ore.Smelt();
    }
};

#endif // OOP_IRONSTACK_H
```

Class Template

- 為什麼 `typename T` 會知道有 `Smelt()` 這個 function?
- 或者說, `Stack` 要怎麼確保丟進來的型別都有 `Smelt()` 這個 function?

```
#ifndef OOP_STACK_H
#define OOP_STACK_H

#include <vector>
#include <string>

template <typename T>
class Stack {
private:
    std::vector<T> vec;
public:
    void push(T ore){
        vec.push_back(ore);
    }
    void pop(){
        vec.pop_back();
    }
    int size(){
        return vec.size();
    }
    std::string smelt(){
        T ore = vec.back();
        vec.pop_back();
        return ore.Smelt();
    }
};

#endif // OOP_STACK_H
```

Class Template

- 為什麼 `typename T` 會知道有 `Smelt()` 這個 function?
- 或者說, `Stack` 要怎麼確保丟進來的型別都有 `Smelt()` 這個 function?
- 事實上, `template` 對於函數的綁定是靜態綁定 (Early Binding), 因此對於每一個 `template` 型別的變數, 他們都會自動去綁定該傳入型別的函數
- `Template` 的 function 實作一定要寫在 header 上

```
#ifndef OOP_STACK_H
#define OOP_STACK_H

#include <vector>
#include <string>

template <typename T>
class Stack {
private:
    std::vector<T> vec;
public:
    void push(T ore){
        vec.push_back(ore);
    }
    void pop(){
        vec.pop_back();
    }
    int size(){
        return vec.size();
    }
    std::string smelt(){
        T ore = vec.back();
        vec.pop_back();
        return ore.Smelt();
    }
};

#endif // OOP_STACK_H
```

Class Template

- 事實上, template 對於函數的綁定是靜態綁定 (Early Binding), 因此對於每一個 template 型別的變數, 他們都會自動去綁定該傳入型別的函數。
- For example?

```
#include "SmeltableOre.hpp"

class IronOre : public SmeltableOre{
public:
    IronOre() : SmeltableOre( result: "Iron Ingot"){

    }
};
```



我們可以確保 IronOre
有 Smelt() 的定義 (繼承)

```
#ifndef ORE_HPP
#define ORE_HPP

#include <string>

class SmeltableOre {
private:
    /// 燒製的結果
    std::string smeltResult;
public:
    SmeltableOre(std::string result){
        this->smeltResult = result;
    }

    /// 燒製並得到錠
    virtual std::string Smelt(){
        return this->smeltResult;
    }
};

#endif
```

Class Template

- 事實上, template 對於函數的綁定是靜態綁定 (Early Binding), 因此對於每一個 template 型別的變數, 他們都會自動去綁定該傳入型別的函數。
- 實際上會綁定 furnace 的 ore.Smelt() 會綁定 GoldOre 的 Smelt() 函數。

```
#include <iostream>

#include "../include/IronOre.hpp"
#include "../include/Stack.h"

int main(){
    Stack<IronOre> stack;

    stack.push( ore: IronOre());
    std::cout << stack.smelt() << std::endl;
}
```



Early binding

```
#ifndef OOP_STACK_H
#define OOP_STACK_H

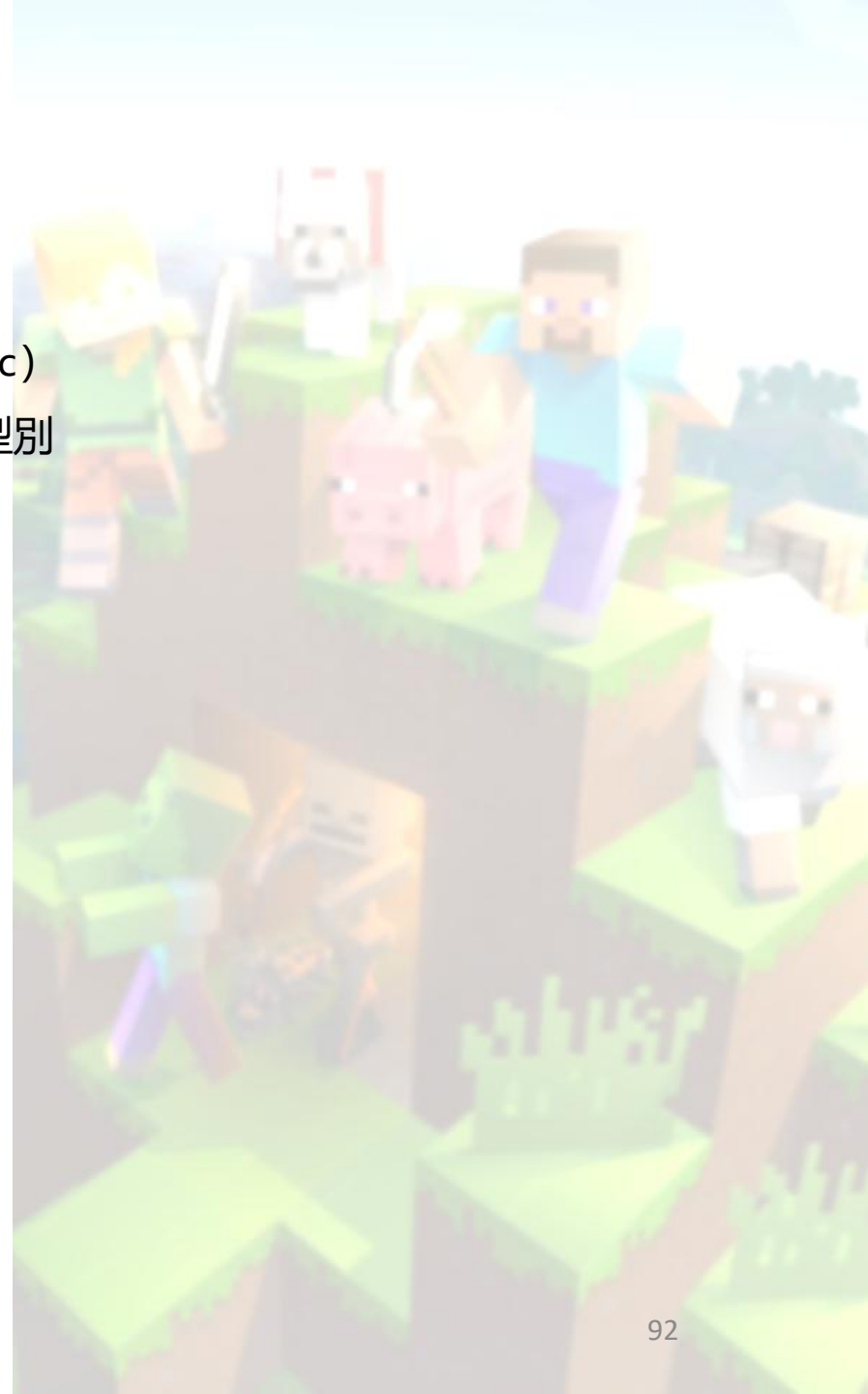
#include <vector>
#include <string>

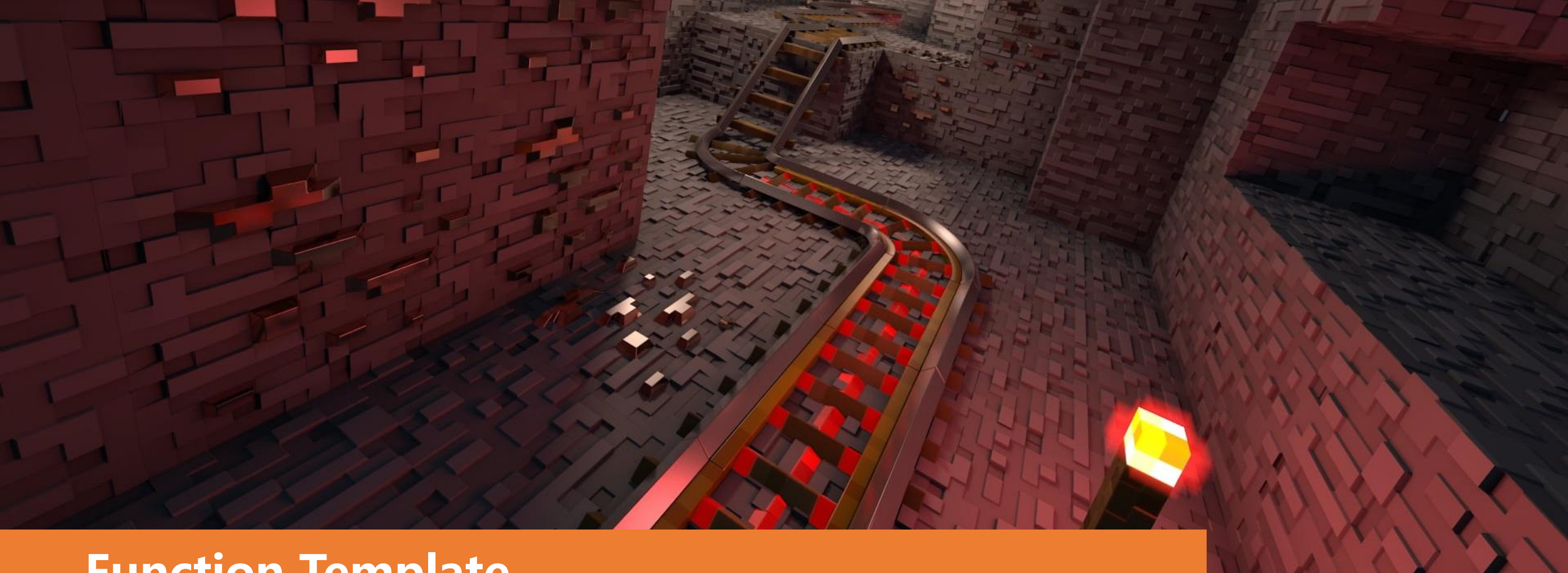
template <typename T>
class Stack {
private:
    std::vector<T> vec;
public:
    void push(T ore){
        vec.push_back(ore);
    }
    void pop(){
        vec.pop_back();
    }
    int size(){
        return vec.size();
    }
    std::string smelt(){
        T ore = vec.back();
        vec.pop_back();
        return ore.Smelt();
    }
};

#endif // OOP_STACK_H
```

Class Template

- 透過 Class Template, 我們可以實現將 Class 內部的型態是通用的 (Generic)
- 使用 Class Template 的類別, 可以傳入特定型別, 並替換通用型別變成指定型別



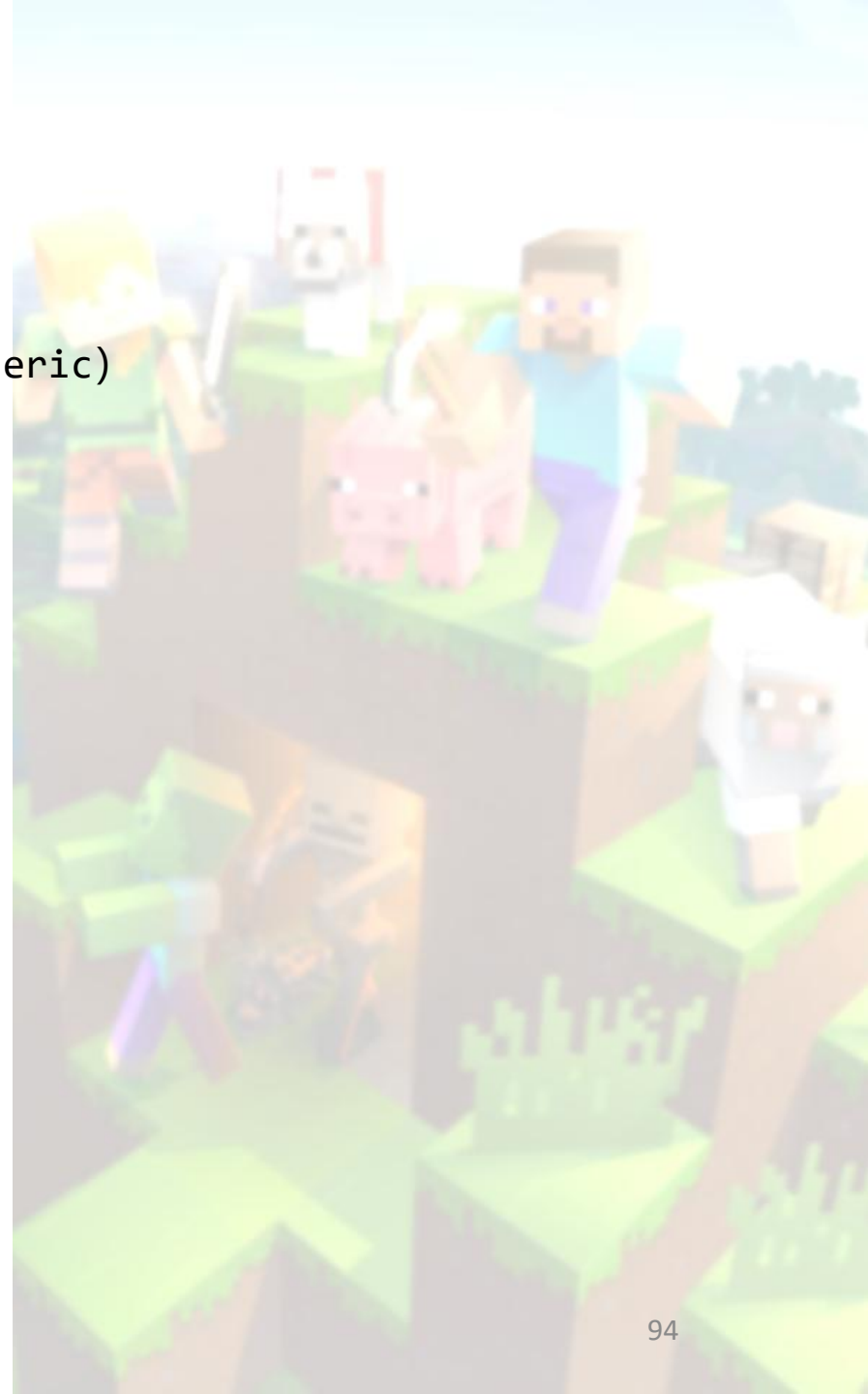


Function Template

- 我們將解釋 Function 如何使用樣板的概念來達成 Parametric 多型

Function Template

- Function Template 主要來說可以幫助我們讓某個函數可以使用通用型別 (Generic)
- 他與 Class Template 沒有不同的差別，只有在設計方法上有些差別



Function Template

- 接下來我們考慮在 Stack 上實作一個 smelt_many 的 function, 目標達到批量燒製礦物
- 這個 smelt_many 的 function 可以:
 - 支援你傳入任意一個容器的型別
 - 傳入你想要 smelt 的數量

```
#ifndef OOP_STACK_H
#define OOP_STACK_H

#include <vector>
#include <string>

template <typename T>
class Stack {
private:
    std::vector<T> vec;
public:
    void push(T ore){
        vec.push_back(ore);
    }
    void pop(){
        vec.pop_back();
    }
    int size(){
        return vec.size();
    }
    std::string smelt(){
        T ore = vec.back();
        vec.pop_back();
        return ore.Smelt();
    }
};

template <typename VT>
VT smelt_many(int n){
    VT container = VT();
    for(int i = 0; i < n; i++){
        container.push_back(smelt());
    }
    return container;
};

#endif // OOP_STACK_H
```

Function Template

- 就會像這樣。

```
#include <iostream>

#include "../include/IronOre.hpp"
#include "../include/Stack.h"

int main() {
    Stack<IronOre> stack;

    stack.push(ore: IronOre());
    stack.push(ore: IronOre());
    stack.push(ore: IronOre());
    stack.push(ore: IronOre());
    stack.push(ore: IronOre());

    std::vector<std::string> vec = stack.smelt_many<std::vector<std::string>>(n: 3);

    for(std::string s : vec){
        std::cout << s << std::endl;
    }

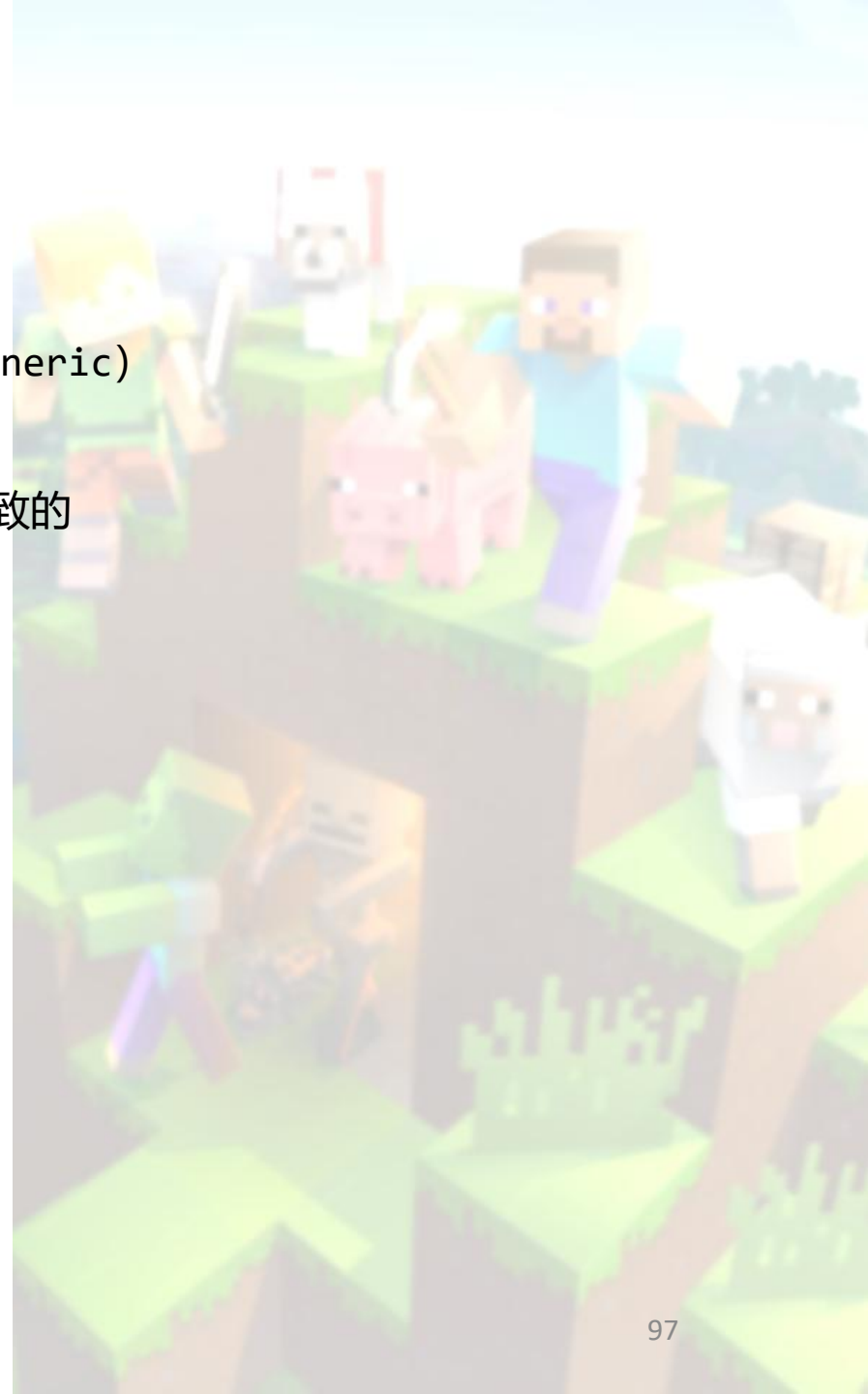
    std::cout << stack.size() << std::endl;
}
```

```
/Users/xuan/Documents/00P2023f_example_code/main
Iron Ingot
Iron Ingot
Iron Ingot
2

Process finished with exit code 0
```

Function Template

- 透過 Function Template, 我們可以對特定 Function 使它擁有通用型別 (Generic)
- 一般來說 Function Template 與實際上 Class Template 運作原理應該是一致的
- 但你可以視情況來決定應該用哪種設計方法



補充資料

一般函數定義

```
void MySwap(int& x, int& y)
{
    int T;
    T = x;
    x = y;
    y = T;
}
```

1. 先設計一個一般函數，測試功能無誤。
2. 標示將來要替換成不同資料類型處。

- 目前只能處理輸入是 INT 型別
- 如何支援輸入是 Float 或是 Char 的型別？



補充資料

一般函數定義

```
void MySwap(int& x, int& y)
{
    int T;
    T = x;
    x = y;
    y = T;
}
```

1. 先設計一個一般函數，測試功能無誤。
2. 標示將來要替換成不同資料類型處。

函數樣版定義

```
template <typename R>
void MySwap(R& x, R& y)
{
    R T;
    T = x;
    x = y;
    y = T;
}
```

1. 函數上方加入 `template <typename R>`
2. 將不同資料類型處改成定義的型態名稱。

補充資料

```
template <typename R>
void MySwap(R& x, R& y)
{
    R T;
    T = x;
    x = y;
    y = T;
}
```

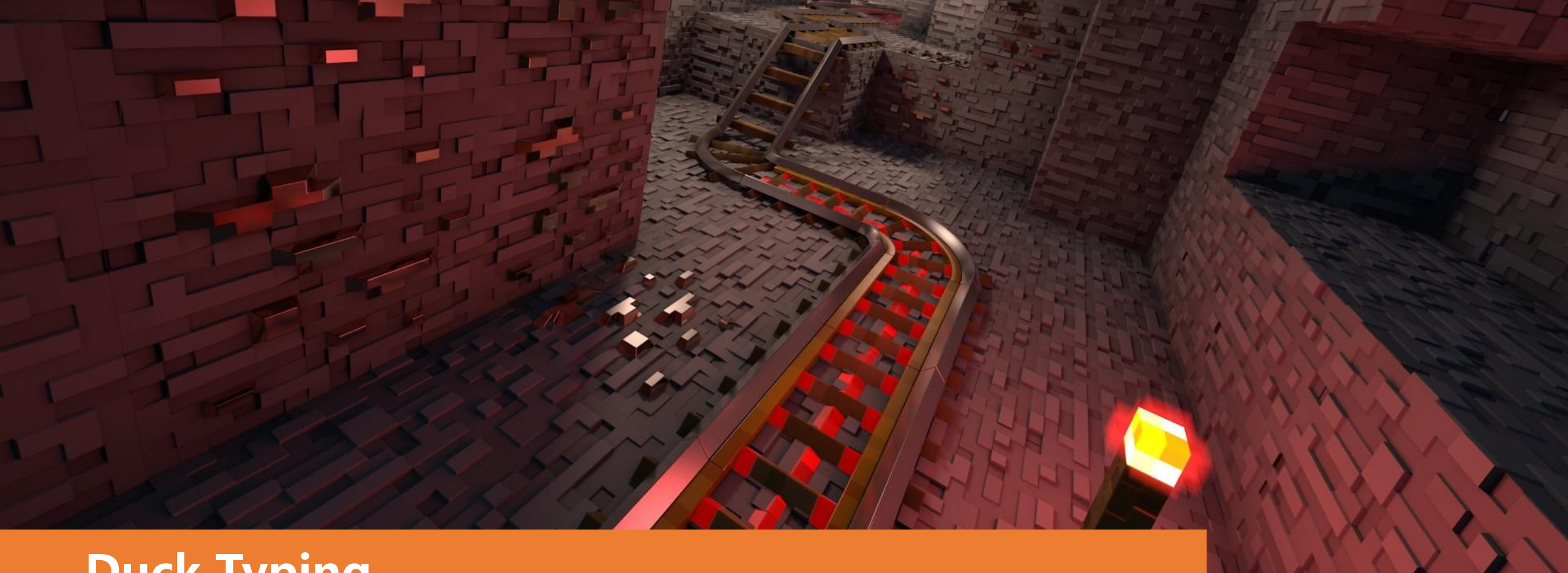
1.主程式

```
...
char a='A';
char b='B';
MySwap(a,b);
...
```

template 樣版關鍵字，用參數化的方式，來定義函數樣版內的資料型態。

<typename R> 宣告底下函數裏面的 **R** 是一個資料型態，將來會由引數的資料型態取代。

R 會被呼叫者傳入參數的資料型態所取代，例如 **int**、**float**、**char**、**string** 等。



Duck Typing

- If it looks like a duck and quacks like a duck, it must be a duck

Duck Typing

- 什麼叫做 Duck Typing?
 - 如果牠看起來像隻鴨子，叫起來像隻鴨子，那牠就一定是隻鴨子
 - 基本上只要你能叫，你能在水中滑行，我們可以不用管你是不是真的鴨子
- 假設我們有一個鴨子的類別
 - 他可以呱呱叫 (Quark)
 - 他可以在水中滑行 (Slide)
- 假設我們有一個「人」的類別
 - 她/他可以學鴨子呱呱叫 (Quark)
 - 她/他可以透過仰式在水中滑行 (Slide)
- 「人的類別」其實可以被丟到「支援鴨子相關操作的泛形」中，也就是 Duck Typing



Duck Typing

- 回顧一下 Stack 中的 smelt() 函數
 - 他必須要可以被燒製 (Smelt) 並且回傳一個 std::string 的產物
 - 如果不能的話他應該會 Compilation Error
- 直覺上我們應該是只放礦物，事實上我們只能放礦物嗎？
 - 你可以想像一下如果「木頭」是可以被燒製的 (Smelt)
 - 然後燒完會得到木炭
 - 那麼木頭其實也可以放到 T 中，畢竟它是可以支援 Smelt() 的
- 所以實際上放入 T 的東西不一定要是 Smelttable
 - 只要可以支援這個 template 類別所需要的最小要求都可以
 - 這就是 duck typing
 - 我們可以不用管你是不是真的礦，但你要能被燒

```
#ifndef OOP_STACK_H
#define OOP_STACK_H

#include <vector>
#include <string>

template <typename T>
class Stack {
private:
    std::vector<T> vec;
public:
    void push(T ore){
        vec.push_back(ore);
    }
    void pop(){
        vec.pop_back();
    }
    int size(){
        return vec.size();
    }
    std::string smelt(){
        T ore = vec.back();
        vec.pop_back();
        return ore.Smelt();
    }
};

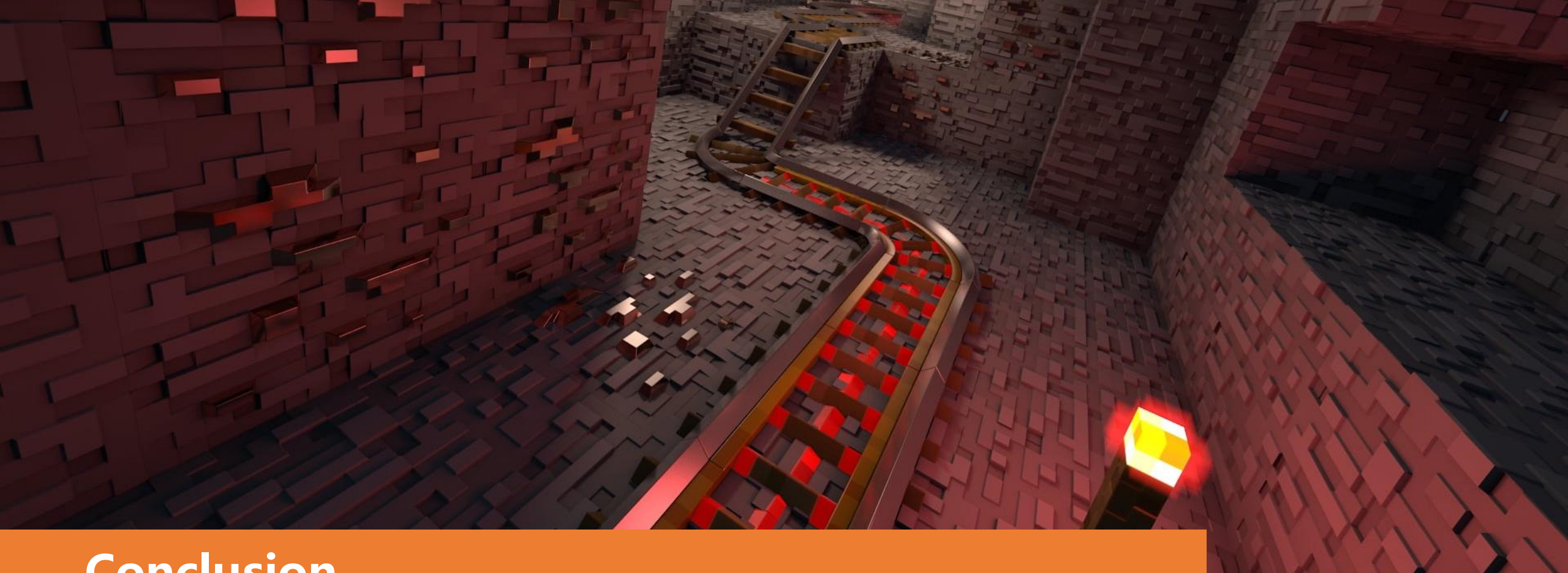
template <typename VT>
VT smelt_many(int n){
    VT container = VT();
    for(int i = 0; i < n; i++){
        container.push_back(smelt());
    }
    return container;
};

#endif // OOP_STACK_H
```

Duck Typing 造成的問題

- 我們期望鴨子，走起來像鴨子與叫起來像鴨子，但現實中他可能不完全是真正的鴨子
 - 實際上你可以有一個「北科大資工系的學生老師」走起來像鴨子且叫起來像鴨子
 - 實際上你可以有一個「唐老鴨造型的玩家」走起來像鴨子且叫起來像鴨子
- 但你可能不期望這些仿鴨子的生物能夠進入鴨子池，成為了濫用 `template` 的例子
 - 如果你不期望這件事情發生，你應該使用繼承多型或介面
- 因此，在 `template` 的使用與設計上，可能要多加注意使用情境





Conclusion

- 結論!

Conclusion

- 透過多型，我們可以盡可能不考慮「精確的型態」這件事情，而是使用繼承多型或泛型來解決型態上的問題
- 另外我們也討論到了一些有趣的議題：
 - Virtual Function 要怎麼去取得底下的實作？
 - 要怎麼善用 Object Casting 來從子類變成父類型別？
 - 為什麼我們會需要 Virtual Destructor？
 - 什麼叫做 Duck Typing？
- 透過這些有趣的議題，使我們能夠解決更多實作面上的問題



兩個 OOP 助教之間的奇妙談話